SRI SHAKTHI

INSTITUTE OF ENGINEERING AND TECHNOLOGY

COIMBATORE – 641062

Autonomous Institution, Accredited by NAAC with 'A' Grade

POWERING THE YOUTH
EMPOWERING THE NATION

21CS514 – WEB DEVELOPMENT LABORATORY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

EX NO	DATE	TOPIC
1.		INTERACTIVE PRICING SLIDER
2.		RESPONSIVE IMAGE GALLERY
3.		WEBPAGE USING GRID LAYOUT
4.		LOGIN FORM VALIDATION USING JS
5.		WEBPAGE USING JS
6.		WEBPAGE USING JS USING ASYCHRONOUS FUNCTION
7.		WEBPAGE USING REACT
8.		WEBPAGE USING NODE,EXPRESS AND REACT
9.		HANDLE EXCEPTIONS IN REACT APPLICATION USING TRY...CATCH
10.		MANIPULATE DOM ELEMENTS DYNAMICALLY IN REACT
11.		CREATE JSX-SYNTAX BASED APPLICATION USING JSX
12.		HANDLE USER EVENTS IN REACT COMPONENTS
13.		IMPLEMENT CONDITIONAL RENDERING IN A REACT APP
14.		RENDER LISTS WITH UNIQUE KEYS FROM DATA
15.		MANAGE COMPONENT STATE AND LIFECYCLE USING HOOKS
16.		INSTALLATION AND SETUP OF NODE.JS, NPM AND EXPRESS.JS FRAMEWORK
17.		BASIC EXPRESS APP SETUP
18.		IMPLEMENT COOKIE-BASED AUTHENTICATION
19.		SESSION MANAGEMENT USING EXPRESS- SESSION
20.		(A)USER AUTHENTICATION USING PASSPORT.JS
		(B)ADMIN DASHBOARD FOR STUDENT RECORD
21.		JWT AUTHENTICATION

22.		(A)HTML5 CLIENT-SIDE FORM VALIDATION FOR USER REGISTRATION
		(B)CLIENT-SIDE FORM VALIDATION USING JAVASCRIPT
23.		(A)SERVER-SIDE VALIDATION USING EXPRESS.JS & JOI
		(B)SERVER-SIDE VALIDATION USING EXPRESS-VALIDATOR
24.		(A)REAL TIME STUDENT PROFILE PREVIEW WITH ANGULARJS
		(B)REAL TIME STUDENT ATTENDANCE VIEWER
25.		(A)CUSTOM ERROR HANDLING MIDDLEWARE IN EXPRESS
		(B)ANGULARJS FORM VALIDATION
26.		INTEGRATION WITH OPENWEATHER API
27.		(A)SENDING EMAIL WITH NODEMAILER
		(B)HR NOTIFICATION POPUPS IN EMPLOYEE PORTAL
28.		(A)CREATE AND TEST REST API USING POSTMAN
		(B)UNIVERSITY DEPARTMENT LISTING VIA RESTAPI
29.		TOKEN BASED AUTHORIZATION WITH EMAIL
30.		(A)SECURE LOGIN FLOW USING EMAIL VERIFICATION AND JWT
		(B)MULTI LANGUAGE STUDENT PORTAL INTERFACE
31.		DB CONNECTIVITY IN MERN STACK
32.		CRUD OPERATION USING MONGODB

EX NO: 01

DATE:

INTERACTIVE PRICING SLIDER

AIM:

Create a responsive pricing slider that allows users to select a pricing plan dynamically, and the displayed price changes accordingly.

ALGORITHM:

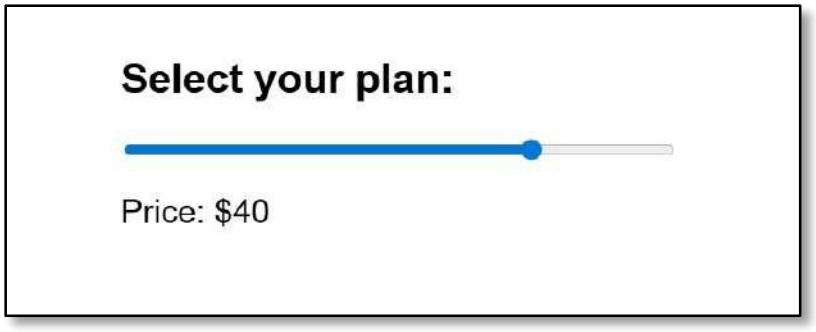
1. Create a range input (slider) in HTML.
2. Set minimum, maximum, and step values according to the pricing tiers.
3. Add an event listener to detect slider changes.
4. Update the displayed price dynamically when the slider is moved.
5. Style the slider and display area for better UX.

PROGRAM :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Interactive Pricing Slider</title>
<style>
  body { font-family: Arial, sans-serif; padding: 20px; }
  .slider-container { max-width: 400px; margin: 0 auto; }
  .price { font-size: 24px; margin: 20px 0; }
  input[type=range] { width: 100%; }
</style>
</head>
<body>
<div class="slider-container">
  <h2>Select your plan:</h2>
  <input type="range" min="1" max="5" value="3" id="priceSlider" />
```

```
  <div class="price">Price: $<span id="priceDisplay">30</span></div>
</div>
<script>
  const priceSlider = document.getElementById('priceSlider');
  const priceDisplay = document.getElementById('priceDisplay');
  const prices = {
    1: 10,
    2: 20,
    3: 30,
    4: 40,
    5: 50
  };
  priceSlider.addEventListener('input', function() {
    priceDisplay.textContent = prices[this.value];
  });
</script></body></html>
```

OUTPUT:



RESULT :

A slider that dynamically updates the price displayed as the user moves the slider thumb.

EX NO: 02

DATE:

RESPONSIVE IMAGE GALLERY

AIM:

Design a responsive image gallery that adjusts the number of columns based on the screen size.

ALGORITHM:

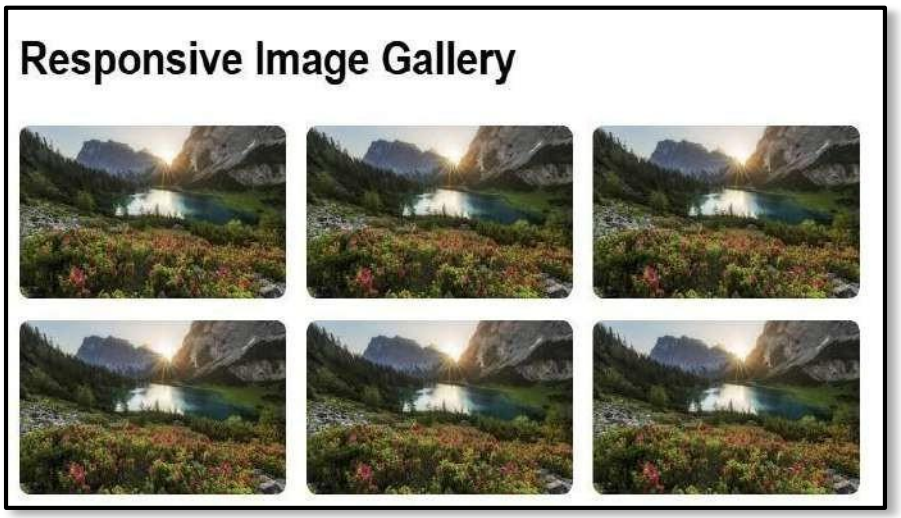
1. Use CSS Grid or Flexbox to layout images.
2. Add media queries for responsiveness.
3. Load multiple images in a container.
4. Adjust columns dynamically with CSS for different screen widths.

PROGRAM :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Responsive Image Gallery</title>
<style>
  body { font-family: Arial, sans-serif; margin: 20px; }
  .gallery {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 15px;
  }
  .gallery img {
    width: 100%;
    height: auto;
    border-radius: 8px;
  }
</style>
```

```
</head> <body>
<h2>Responsive Image Gallery</h2>
<div class="gallery">
  <img src="https://placekitten.com/300/200" alt="Kitten 1" />
  <img src="https://placekitten.com/301/200" alt="Kitten 2" />
  <img src="https://placekitten.com/300/201" alt="Kitten 3" />
  <img src="https://placekitten.com/302/200" alt="Kitten 4" />
  <img src="https://placekitten.com/300/202" alt="Kitten 5" />
  <img src="https://placekitten.com/303/200" alt="Kitten 6" />
</div> </body> </html>
```

OUTPUT



RESULT:

A responsive image gallery that adjusts the number of columns based on screen size, ensuring images look neat on all devices.

EX NO: 03

DATE:

WEBPAGE USING GRID LAYOUT

AIM:

To Create a simple webpage layout using CSS Grid.

ALGORITHM:

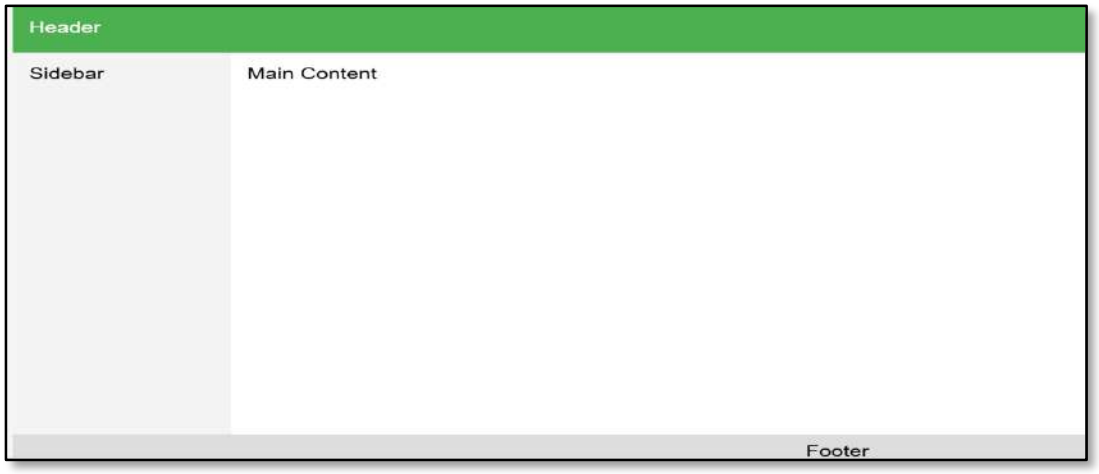
1. Define a grid container in CSS.
2. Set the number of rows and columns.
3. Place header, sidebar, main content, and footer inside grid areas.
4. Style each grid area distinctively.

PROGRAM :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Grid Layout Webpage</title>
<style>
  body { margin: 0; font-family: Arial, sans-serif; }
  .grid-container {
    display: grid;
    grid-template-areas:
      'header header header'
      'sidebar main main'
      'footer footer footer';
    grid-template-columns: 200px 1fr 1fr;
    grid-template-rows: 60px 1fr 40px;
    height: 100vh;  }
  header { grid-area: header; background: #4CAF50; color: white; padding: 15px; }
  sidebar { grid-area: sidebar; background: #f4f4f4; padding: 15px; }
  main { grid-area: main; padding: 15px; }
```

```
footer { grid-area: footer; background: #ddd; text-align: center; padding: 10px; }
</style>
</head>
<body>
<div class="grid-container">
  <header>Header</header>
  <main>Main Content</main>
  <footer>Footer</footer>
</div>
</body>
</html>
```

OUTPUT:



RESULT:

A webpage with a header, sidebar, main content, and footer arranged using CSS Grid layout.

EX NO: 04

DATE:

LOGIN FORM VALIDATION USING JS

AIM:

Validate user input in a login form using JavaScript to ensure the username and password are entered correctly.

ALGORITHM:

1. Create an HTML login form with username and password fields.
2. Use JavaScript to listen for the form submission event.
3. Check if the username and password fields are not empty.
4. If validation fails, prevent form submission and show error messages.
5. If valid, allow form submission or display success.

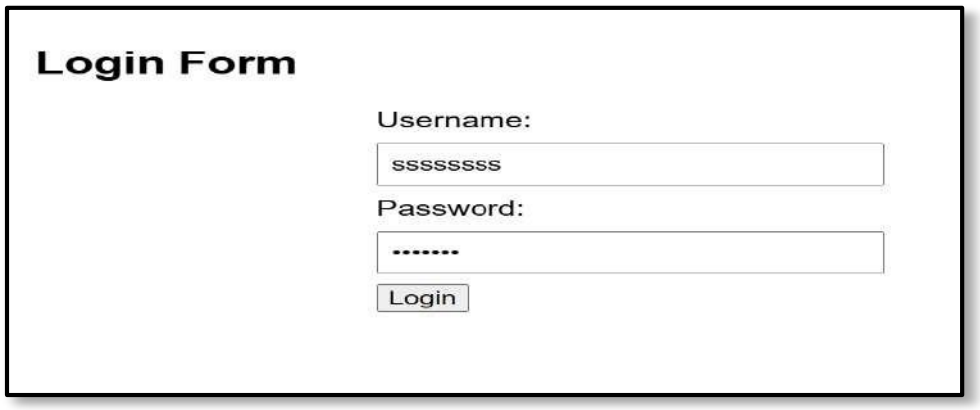
PROGRAM :

```
<!DOCTYPE html>
<html lang=”en”>
<h2>Login Form</h2>
<form id=”loginForm” novalidate>
  <label for=”username”>Username:</label>
  <input type=”text” id=”username” name=”username” />
  <div id=”usernameError” class=”error”></div>
  <label for=”password”>Password:</label>
  <input type=”password” id=”password” name=”password” />
<script>
  const form = document.getElementById(‘loginForm’);
const username = document.getElementById(‘username’);
let valid = true;
   usernameError.textContent =
   ‘’; passwordError.textContent
   = ‘’;
    if (username.value.trim() === ‘’) {
```

```
usernameError.textContent = ‘Username is required’; valid = false;}

  if (password.value.trim() === ‘’) {
    passwordError.textContent = ‘Password is required’;
    valid = false;
  } else if (password.value.length < 6) {
    passwordError.textContent = ‘Password must be at least 6 characters’;
    valid = false}
  if (!valid) {
    event.preventDefault(); // Prevent form submission if validation fails
  }
});</script></body></html>
```

OUTPUT:



RESULT:

Login Form Validation done Using Js and Output is Verified.

EX NO: 05

DATE:

WEBPAGE USING JS

AIM:

Create a webpage that fetches data from a public API and displays it using Promises without async/await.

ALGORITHM:

1. Use the fetch API and handle the Promise with .then().
2. Convert the response to JSON.
3. Display the data dynamically on the webpage.
4. Handle errors with .catch().

PROGRAM :

```
<!DOCTYPE html>
<html lang=”en”>
<head>
<meta charset=”UTF-8” />
<meta name=”viewport” content=”width=device-width, initial-scale=1” />
<title>Fetch Example without async/await</title>
<h2>Random User Data</h2>
<button id=”fetchBtn”>Fetch User</button>
<ul id=”userList”></ul>
<script>
  const fetchBtn = document.getElementById(‘fetchBtn’);
  const userList = document.getElementById(‘userList’);
  function fetchUser() {
  fetch(‘https://randomuser.me/api/’)
  .then(response => {
       if (!response.ok) {
         throw new Error(‘Network response was not ok’) }
```

```
    })
      const user = data.results[0];
      userList.innerHTML = `
        <li>Name: ${user.name.first} ${user.name.last}</li>
        <li>Email: ${user.email}</li>
        <li>Location: ${user.location.city}, ${user.location.country}</li> })
     .catch(error => {
      userList.innerHTML = 'Failed to fetch data.';
      console.error('Fetch error:', error);
     });
  }
  fetchBtn.addEventListener('click',  fetchUser);
</script> </body> </html>
```

OUTPUT:



RESULT:

Webpage created using JS Successfully

EX NO: 06

DATE:

WEBPAGE USING ASYNCHRONOUS JS

AIM:

Demonstrate asynchronous behavior using JavaScript async functions with a delay.

ALGORITHM:

1. Create an async function that waits for a delay using setTimeout wrapped in a Promise.
2. Display messages before and after the delay.
3. Show how async functions don't block UI.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Async Timeout Example</title>
<style>
  body { font-family: Arial, sans-serif; padding: 20px; }
  #message { margin-top: 20px; font-weight: bold; }
</style>
</head>
<body>

<h2>Async Timeout Demo</h2>
<button id="startBtn">Start</button>
<div id="message"></div>
<script>
  const startBtn = document.getElementById('startBtn');
```

```
  const message = document.getElementById('message');
  function delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
async function asyncDemo() {
    message.textContent = 'Process started…';
    await delay(3000); // wait 3 seconds
    message.textContent = 'Process completed after 3 seconds!';
  }
  startBtn.addEventListener('click',  asyncDemo);
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Webpage created using Asynchronous JS.

EX NO: 07

DATE:

WEBPAGE USING REACT

AIM:

Create a React webpage that displays a greeting message.

ALGORITHM:

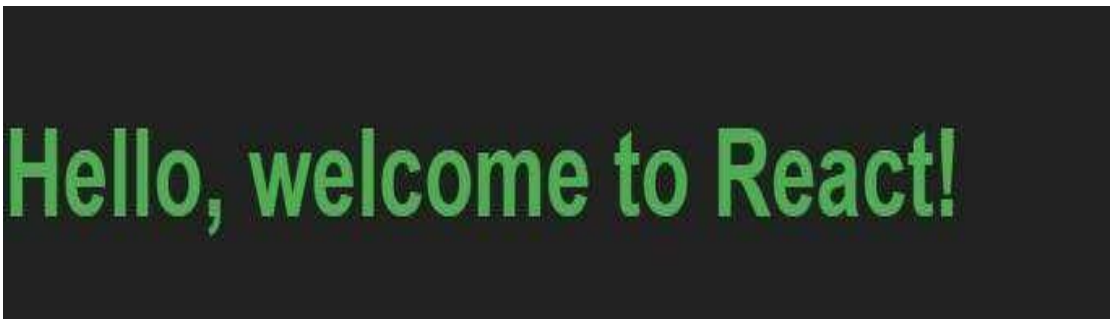
1. Setup React environment (using CDN or create-react-app).
2. Create a functional component.
3. Render the component in the DOM.
4. Display a message dynamically.

PROGRAM:

```
<!DOCTYPE html>
<html lang=”en”>
<head>
<meta charset=”UTF-8” />
<meta name=”viewport” content=”width=device-width, initial-scale=1” />
<title>React Example</title>
<script crossorigin src=”https://unpkg.com/react@18/umd/react.development.js”></script>
<script crossorigin src=”https://unpkg.com/react-dom@18/umd/react-
dom.development.js”></script>
<script crossorigin src=”https://unpkg.com/@babel/standalone/babel.min.js”></script>
</head>
<body>
```

```
<div id=”root”></div>
<script type=”text/babel”>
  function Greeting() {
    return <h1>Hello, welcome to React!</h1>;
  }
  const root = ReactDOM.createRoot(document.getElementById(‘root’));
  root.render(<Greeting />);
</script>
</body>
</html>
```

OUTPUT:



RESULT:

A Simple Page Created Using React and Output is Verified.

EX NO: 08

DATE:

WEBPAGE USING NODE, EXPRESS AND REACT

AIM:

Build a simple full-stack app with Node.js backend (Express) and React frontend.

ALGORITHM:

1. Setup Node.js and Express server that serves data or frontend files.
2. Create a React app (frontend).
3. Connect frontend to backend via API calls.
4. Display data fetched from backend in React component.

PROGRAM (Basic setup snippets):

Backend (Node.js + Express) — server.js

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/api/message', (req, res) => {
  res.json({ message: 'Hello from Express backend!' });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run with node server.js.

Frontend (React) — App.js

```
import React, { useEffect, useState } from 'react';

function App() {
  const [message, setMessage] = useState('');
  useEffect(() => {
```

```
 fetch('/api/message')
      .then(res => res.json())
.then(data => setMessage(data.message))
      .catch(err => setMessage('Error fetching data'));
  }, []);
  return (
    <div>
      <h1>Message from backend:</h1>
      <p>{message}</p>
    </div>
  );
}
export default App;
```

OUTPUT:



RESULT:

Webpage using node, express and react Successfully.

EX NO: 09

DATE:

HANDLE EXCEPTIONS IN REACT APPLICATION USING TRY...CATCH

AIM:

To handle exceptions in react application using try...catch.

ALGORITHM:

1. Start the React component and initialize state (data, error).
2. Run risky code (API call or logic) inside a try block.
3. If code succeeds, update state with the result.
4. If an error occurs, catch it and update state with the error message.
5. Render UI → show either the result or the error message.

PROGRAM:

```
import React, { useState } from "react";
function App() {
const [message, setMessage] = useState("");
const handleClick = () => {
        try {
          let result = notDefinedVar + 10;
          console.log(result);
        } catch (err) {
          setMessage("Error: " + err.message);
        }
      };
      return (
        <div style={{ fontFamily: "Arial", padding: "20px" }}>
          <h1>React Exception Handling (try...catch) </h1>
          <button onClick ={handleClick}>Click Me </button>
```

```
{message && <p style={{ color: "red" }}>{message}</p>}
    </div>
  );
}
export default App;
```

OUTPUT:



RESULT:

Thus, the exception handling in react application was executed.

EX NO: 10

DATE:

MANIPULATE DOM ELEMENTS DYNAMICALLY IN REACT

AIM:

To manipulate DOM elements dynamically in react.

ALGORITHM:

1. Start the React app and initialize a state variable show = true.
2. Display a button with text "Hide Text" if show is true, else "Show Text".
3. Check button click → when clicked, update the state show = !show (toggle value).
4. If show is true → display the <p> element "Hello, I appear and disappear dynamically!".
5. If show is false → remove the <p> element from the DOM.

PROGRAM:

```
import React, { useState } from "react";
function App() {
const [show, setShow] = useState(true);
return (
<div style={{ padding: "20px", fontFamily: "Arial" }}>
 <h1>DOM Manipulation in React</h1>
 <button onClick={() => setShow(!show)}>
 {show ? "Hide Text" : "Show Text"}
 </button>
 {show && <p>Hello, I appear and disappear dynamically!</p>}
 </div>
      );
     }
     export default App;
```

OUTPUT:



RESULT:

Thus,the DOM Manipulation in react was executed successfully.

EX NO: 11

DATE:

CREATE JSX-SYNTAX BASED APPLICATION USING JSX

AIM:

To Create JSX-syntax based application using JSX.

ALGORITHM:

1. Start the React app and initialize state variable count = 0.
2. Render UI → display a heading, a paragraph, and a button showing Clicked 0 times.
3. Wait for user action → if the button is clicked, trigger the onClick event.
4. Update state → increase count by 1 (count = count + 1).
5. Re-render UI → show the updated button text "Clicked {count} times".

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
 <head>
   <meta charset="UTF-8" />
   <title>JSX Example</title>
   <script  crossorigin  src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    </head> <body>
   <div id="root"></div>
 <script type="text/babel">
function App() {
   const [count, setCount] = React.useState(0);
return (
    <div>
 <h1>JSX Syntax Example</h1>
```

```
    <p>Hello, this text is written in <strong>JSX</strong>!</p>
          <button onClick={() => setCount(count + 1)}>
            Clicked {count} times
          </button>
        </div>
      );
    }
    const root = ReactDOM.createRoot(document.getElementById("root"));
  </script>
 </body>
</html>
```

OUTPUT:

JSX Syntax Example

Hello, this text is written in **JSX**!

Clicked 3 times

RESULT:

Thus, the create JSX-syntax based application using JSX was displayed.

EX NO: 12

DATE:

HANDLE USER EVENTS IN REACT COMPONENTS

AIM:

To handle user events in react components.

ALGORITHM:

1. Initialize component with two states: text (for input) and message (for button click feedback).

2. Render UI with a heading, button, message area, input box, and typed text display.

3. Handle button click using handleClick() → update message to "Button was clicked!".

4. Handle input change using handleChange() → update text with whatever the user types.

5. Update output dynamically → React re-renders to show the latest message and text values.

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>User Events in React</title>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>  <body>
      function App() {
        const [text, setText] = React.useState("");
        const [message, setMessage] = React.useState("");
        const handleClick = () => {
          setMessage("Button was clicked!"); }
       const handleChange = (event) => {
```

```
setText(event.target.value);   };
```

```
return (
        <div style={{ fontFamily: “Arial”, padding: “20px” }}>
          <h1>React User Events</h1>
          <button onClick={handleClick}>Click Me</button>
          <p>{message}</p>
          <input type=”text” placeholder=”Type something…”  value={text}  />
          <p>You typed: {text}</p>
        </div>  );  }
     ReactDOM.createRoot(document.getElementById(“root”)).render(<App />);
   </script>
  </body>
</html>
```

OUTPUT:



RESULT:

Thus, the handle user events in react components was executed.

EX NO: 13

DATE:

IMPLEMENT CONDITIONAL RENDERING IN A REACT APP

AIM:

To implement conditional rendering in a react app.

ALGORITHM:

1. Start the React app and initialize a state variable isLoggedIn as false.
2. Render the heading and a section that changes based on the value of isLoggedIn.
3. If isLoggedIn is true, display a welcome message with a Logout button.
4. If isLoggedIn is false, display a login message with a Login button.
5. Update the state when buttons are clicked and re-render the UI accordingly.

PROGRAM:

```
<!DOCTYPE html>
<html>
 <head>
 <meta charset="UTF-8" />
  <title>Conditional Rendering in React</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  </head>
  <body>
      function App() {
        const [isLoggedIn, setIsLoggedIn] = React.useState(false);
        const handleLogin = () => setIsLoggedIn(true);
        const handleLogout = () => setIsLoggedIn(false);
        return (
          <div style={{ fontFamily: "Arial", padding: "20px" }}>
            <h1>Conditional Rendering Example</h1>
```

```
        {/* Conditional Rendering */}
        {isLoggedIn ? (
          <div>
            <h2>Welcome, User! </h2>
            <button onClick={handleLogout}>Logout</button> </div>
        ) : ( <div>
            <h2>Please log in to continue.</h2>
            <button onClick={handleLogin}>Login</button>
          </div> ) }
      </div> ); }
    ReactDOM.createRoot(document.getElementById("root")).render(<App />);
  </script>
 </body>
</html>
```

OUTPUT:



RESULT:

Thus, the conditional rendering in react was executed.

EX NO: 14

DATE:

RENDER LISTS WITH UNIQUE KEYS FROM DATA

AIM:

To render lists with unique keys from data in react.

ALGORITHM:

1. Initialize a list of objects (users) with unique id values.
2. Use the map() function to loop through the list.
3. For each item, create a React element (<li>).
4. Assign the key attribute using the unique id of each item.
5. Render the list inside a parent container (<ul>).

PROGRAM:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Render Lists in React</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  </head> <body>
      function App() {
      const users = [
          { id: 1, name: "Naveen", age: 22 },
          { id: 2, name: "Sunil", age: 25 },
          { id: 3, name: "Selva", age: 30 } ];
        return (
          <div style={{ fontFamily: "Arial", padding: "20px" }}>
            <h1>Rendering Lists in React</h1>
```

```
        <ul>
          {users.map((user) => (
            <li key={user.id}> {user.name} — Age: {user.age}
            </li>
          ) ) } </ul>
        </div>
      ); }
    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(<App />);
  </script>
 </body>
</html>
```

OUTPUT:



RESULT:

Thus, the render lists with unique keys from data in react was executed.

EX NO: 15

DATE:

MANAGE COMPONENT STATE AND LIFECYCLE USING HOOKS

AIM:

To manage component state and lifecycle using hooks.

ALGORITHM:

1. Initialize state variables using useState (e.g., count, time).
2. Display state values in the UI.
3. Update state when user interacts (e.g., button click changes count).
4. Use useEffect to run lifecycle logic (e.g., start timer on mount).
5. Clean up resources in useEffect return function when component unmounts.

PROGRAM:

```
<!DOCTYPE html>
<html>
 <head>
  <meta charset="UTF-8" />
  <title>React Hooks Example</title>
  <script  src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
 </head> <body>
    function App() {
      const [count, setCount] = React.useState(0);
      const [time, setTime] = React.useState(new Date().toLocaleTimeString());
      React.useEffect(() => {
        console.log("Component mounted ;
        const timer = setInterval(() => {
          setTime(new Date().toLocaleTimeString()); }, 1000);
        return () => { clearInterval(timer);
```

```
      console.log("Component unmounted); }; }, []);
      return (
        <h1>React Hooks: State & Lifecycle</h1>
        <p>Counter: {count}</p>
        <button onClick={() => setCount(count + 1)}>Increment</button>
        <button onClick={() => setCount(count - 1)}>Decrement</button>
        <h2>Current Time: {time} </h2> );   }
    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(<App />);
  </script>
 </body>
</html>
```

OUTPUT:



RESULT:

Thus, the manage component state and lifecycle using hooks was executed.

EXP NO : 16 **INSTALLATION AND SETUP OF NODE.JS , NPM**

DATE : **AND EXPRESS FRAMEWORK**

AIM :

To install node.js and npm using command prompt and setup the project folder in visual studio.

DEFINITION :

NODE.JS

Node.js is an open-source and cross-platform JavaScript runtime environment. It is a popular tool for almost any kind of project. Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser. This allows Node.js to be very performant. Node.js allows you to write JavaScript on both client and server.

NPM

NPM (Node Package Manager) is the default package manager for Node.js, used to install, share, and manage code packages (libraries or modules) for JavaScript projects. It allows developers to easily include reusable code from a vast ecosystem of open-source packages hosted on the npm registry. With NPM, you can manage project dependencies, run scripts, and control project versions using a simple command-line interface.

EXPRESS.JS

Express.js is a fast, minimal, and flexible web application framework for Node.js that simplifies the process of building server-side applications and APIs. It provides robust features such as routing, middleware support, and HTTP utility methods, allowing developers to handle requests, responses, and application logic efficiently. Express is widely used for building RESTful APIs and web applications, making it a core part of the MEAN and MERN stacks. Its simplicity and scalability make it ideal for both small projects and large-scale applications.

STEPS TO INSTALL :

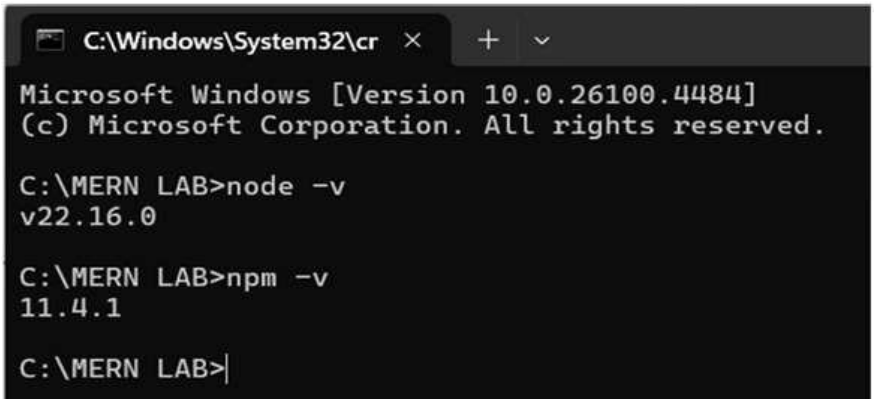
STEP-1

Check if already installed:

Open a terminal and run:

node -v

npm -v

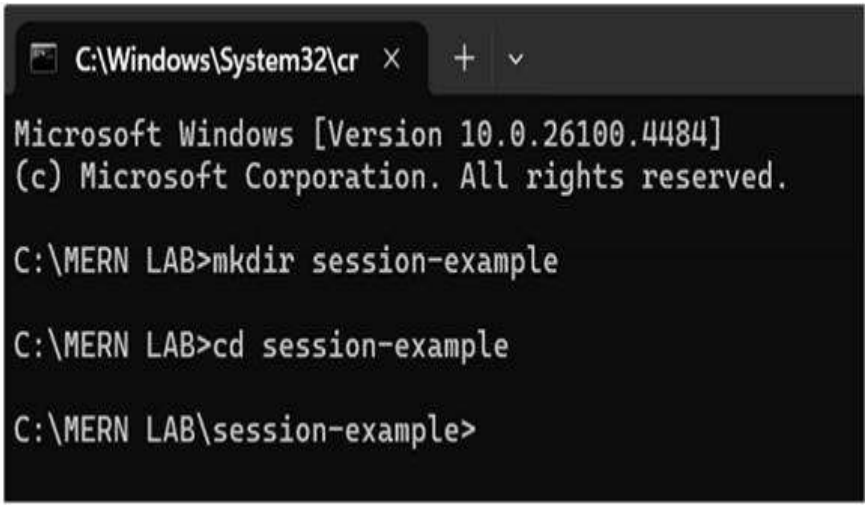


If not installed:

Download & Install from: https://nodejs.org

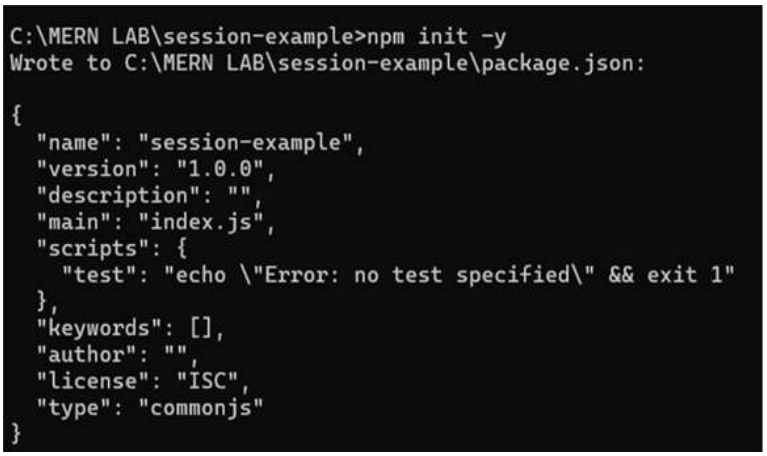
STEP 2:

Create a Project Folder Use your terminal or command prompt:



STEP 3:

Initialize Node.js Project npm init -y This creates a package.json file.



STEP 4:

Install Required npm Packages Run the following command to install: npm install express express-session

```
C:\MERN LAB\session-example>npm install

up to date, audited 1 package in 501ms

found 0 vulnerabilities

C:\MERN LAB\session-example>npm install express

added 67 packages, and audited 68 packages in 3s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\MERN LAB\session-example>npm install express-session

added 7 packages, and audited 75 packages in 2s

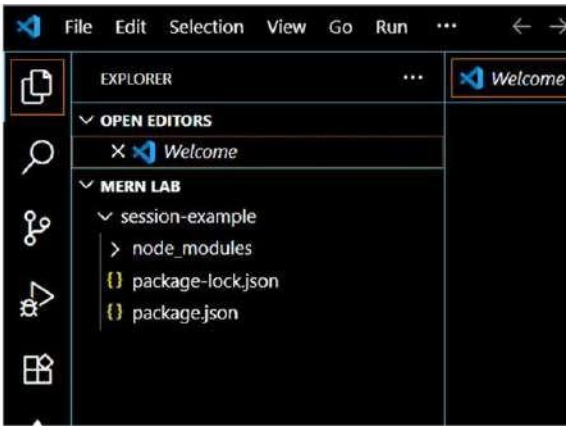
14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\MERN LAB\session-example>
```

STEP 5:

Directory Overview Your folder should now contain:



RESULT :

Thus, the installation of node,js, npm and express.js was successfully completed.

EXP NO : 17

BASIC EXPRESS APP SETUP

DATE :

AIM :

To develop a basic web server using Express.js that simulates a hospital booking homepage with multiple navigation routes. This includes creating routes for the homepage, appointments, doctors, and contact sections. The experiment aims to demonstrate how Express handles simple routing and responses on a local server.

ALGORITHM :

Step 1: Import the Express module using require.

Step 2: Create an Express app using express().

Step 3: Create a route for the homepage (/).

Step 4: Create a route for the appointments page (/appointments).

Step 5: Create a route for the doctors page (/doctors).

Step 6: Create a route for the contact page (/contact).

Step 7: Set a port number (e.g., 3000).

Step 8: Start the server and print a message to show it is running.

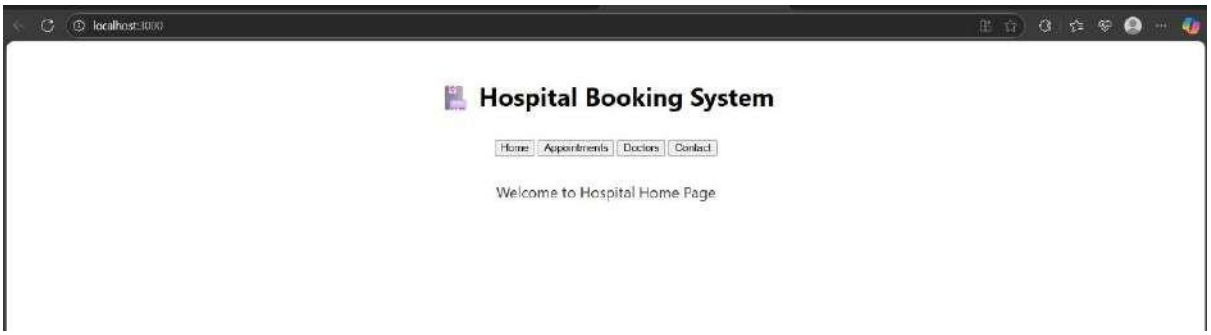
PROGRAM:

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
function Home() {
  return <h2>Welcome to Hospital Home Page</h2>;
}
function Appointments() {
  return <h2> Book Your Appointment</h2>;
}
function Doctors() {
  return <h2> List of Doctors</h2>;
}
```

```
function Contact() {
  return <h2>! Contact Us</h2>;
}
function App() {
  return (
    <Router>
      <div style={{ textAlign: 'center', marginTop: '2rem' }}>
        <h1>Hospital Booking System</h1>
        <nav style={{ marginBottom: '1.5rem' }}>
          <Link to="/" style={{ margin: '1rem' }}>Home</Link>
          <Link to="/appointments" style={{ margin: '1rem' }}>Appointments</Link>
          <Link to="/doctors" style={{ margin: '1rem' }}>Doctors</Link>
          <Link to="/contact" style={{ margin: '1rem' }}>Contact</Link>
        </nav>
<Routes>
          <Route path="/" element={<Home />} />
          <Route path="/appointments" element={<Appointments />} />
          <Route path="/doctors" element={<Doctors />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </div>
    </Router>
  );
}
export default App;
```

OUTPUT:

1. Home Page:



2. Appointment Page:



RESULT:

A basic hospital homepage using Express.js was successfully implemented. The server handles different routes effectively, providing appropriate responses for the homepage, appointments, doctors, and contact sections.

EXP NO : 18 **IMPLEMENT COOKIE BASED AUTHENTICATION**

DATE :

AIM :

To implement a cookie-based login system using Express.js and cookie-parser for a simple shopping cart application. This experiment demonstrates user authentication using cookies to personalize cart access and maintain session-like behavior.

ALGORITHM :

Step 1: Import the required modules – express and cookie-parser.

Step 2: Create an Express app using express().

Step 3: Use middleware to parse URL-encoded request bodies using express.urlencoded().

Step 4: Use middleware to parse cookies using cookie-parser().

Step 5: Create a login route (/login):

Accept the username via a POST request.

Store the username in a cookie using res.cookie().

Step 6: Create a cart route (/cart):

Read the user cookie using req.cookies.

If the cookie exists, display a personalized cart message.

If not, prompt the user to log in.

Step 7: Create a logout route (/logout):

Clear the user cookie using res.clearCookie().

Send a message confirming logout.

Step 8: Start the server on port 3000 using app.listen() and print a message showing the server is running.

PROGRAM:

```
import React, { useState } from 'react';

import { BrowserRouter as Router, Routes, Route, Link, useNavigate } from 'react-router-dom';

function Login({ setUser }) {
```

```
  const [username, setUsername] = useState(‘’);
  const navigate = useNavigate();
const handleLogin = € => {
    e.preventDefault();
    if (username.trim()) {
      setUser(username);
      navigate(‘/cart’);
    }
  };
return (
    <div>
      <h2>Login</h2>
      <form onSubmit={handleLogin}>
        <input
          type=”text”
          placeholder=”Enter username”
          value={username}
          onChange={€ => setUsername(e.target.value)}
          required
        />
        <button type=”submit”>Login</button>
      </form>
    </div>
  );
}
function Cart({ user }) {
  return (
    <div>
      <h2>Cart</h2>
```

```
      <p>{user ? Your Cart, ${user} : 'Please login to view cart'}</p>
    </div>
  );
}
function Logout({ setUser }) {
const navigate = useNavigate();
const handleLogout = () => {
    setUser('');
    navigate('/');
  };
return (
    <div>
      <h2>Logout</h2>
      <p>You have been logged out.</p>
      <button onClick={handleLogout}>Go to Home</button>
    </div>
  );
}
function Home() {
  return <h2>Welcome to Shopping Cart System</h2>;
}
function App() {
  const [user, setUser] = useState('');
 return (
    <Router>
      <div style={{ textAlign: 'center', marginTop: '2rem' }}>
        <h1>Shopping Cart Authentication</h1>
        <nav style={{ marginBottom: '1.5rem' }}>
          <Link to="/" style={{ margin: '1rem' }}>Home</Link>
```

```
        <Link to="/login" style={{ margin: '1rem' }}>Login</Link>
        <Link to="/cart" style={{ margin: '1rem' }}>Cart</Link>
        <Link to="/logout" style={{ margin: '1rem' }}>Logout</Link>
      </nav>
<Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login setUser={setUser} />} />
        <Route path="/cart" element={<Cart user={user} />} />
        <Route path="/logout" element={<Logout setUser={setUser} />} />
      </Routes>
    </div>
  </Router>
 );
}
export default App;
```

OUTPUT :

1. Home Page:



2. Login Page:



3. Cart Page:



4. Logout Page:



RESULT:

The cookie-based login system was successfully implemented using Express and cookie-parser and it stores user data in cookies to personalize the cart and supports login and logout features.

EXP NO : 19 **SESSION MANAGEMENT USING EXPRESS SESSION**

DATE :

AIM :

To implement a session-based login system for a student portal using Express.js and express-session, allowing a user to log in, access a dashboard, and log out with session tracking.

ALGORITHM :

Step 1: Initialize an Express.js application.

Step 2: Set up the session middleware using express-session.

Step 3: Create a login route to accept username and store it in the session.

Step 4: Create a dashboard route to display data only if a user session exists.

Step 5: Create a logout route to destroy the session and end the login state.

Step 6: Start the server and test using browser or Postman.

PROGRAM :

APP.JS

```
// Import required modules
const express = require('express');
const session = require('express-session');
// Create an Express app
const app = express();
// Middleware to parse URL-encoded bodies (form data)
app.use(express.urlencoded({ extended: true }));
// Session middleware configuration
app.use(session({
    secret: 'portal',            // Secret key to sign the session ID cookie
    resave: false,               // Don't save session if unmodified
    saveUninitialized: true      // Save new sessions even if unmodified
```

```
}));
/ Route: POST /login
// Simulates login by saving username to session
app.post('/login', (req, res) => {
    const username = req.body.username;
    if (username) {
        req.session.user = username;
        res.send( Welcome ${username} to Student Portal);
    } else {
        res.send("+ Please provide a username to log in");
    }
});
// Route: GET /dashboard
// Protected route, accessible only if session exists
app.get('/dashboard', (req, res) => {
    if (req.session.user) {
        res.send( ashboard for ${req.session.user});
    } else {
        res.send("⚠ Please login to access the dashboard");
    }
});
// Route: GET /logout
// Ends the session and logs out the user
app.get('/logout', (req, res) => {
    req.session.destroy((err) => {
        if (err) {
            return res.send("+ Error in logging out");
        }
```

```
        res.send("Session ended. Logged out successfully.");
    });
});
// Start the server
app.listen(3000, () => {
    console.log('Student Portal running on http://localhost:3000');
});
```

LOGIN.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Portal Login</title>
</head>
<body style="text-align: center; margin-top: 100px;">
    <h2>Student Portal Login</h2>
    <form method="POST" action="/login">
        <input type="text" name="username" placeholder="Username" required><br><br>
        <input type="password" name="password" placeholder="Password" required><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
```

DASHBOARD.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
</head>
<body style="text-align: center; margin-top: 50px;">
```

```
    <h2>Welcome <span id=”username”>User</span>,</h2>
    <h3>Student Details</h3>
    <p>Register number: 21CS055</p>
    <p>Department: CSE</p>
    <a href=”/logout”>Logout</a>
<script>
        // Set username from query parameter
        const params = new URLSearchParams(window.location.search);
        const username = params.get(“user”) || “User”;
        document.getElementById(“username”).textContent = username;
    </script>
</body>
</html>
```

LOGOUT.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Logout</title>
</head>
<body style=”text-align: center; margin-top: 100px;”>
    <p><strong>You have been logged out.</strong> <a href=”/”>Login again</a></p>
</body>
</html>
```

OUTPUT :

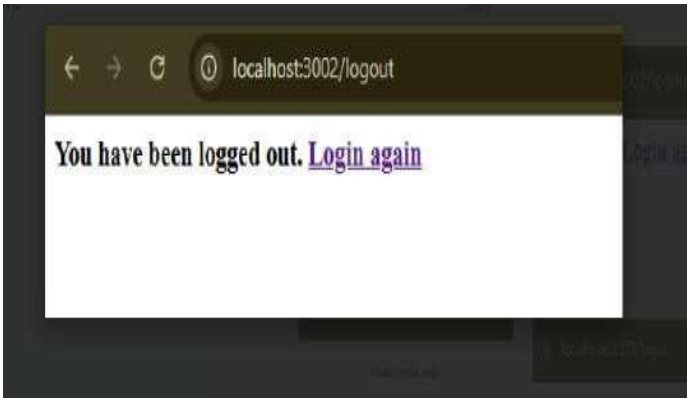
1. Dashboard Page



2. Login Page



3. Logout Page



4. Validate page



RESULT :

The session-based login system for the student portal was successfully implemented using express-session, allowing user session tracking across routes and session termination on logout.

EXP NO : 20(a) **USER AUTHENTICATION USING PASSPOSRT.JS**

DATE :

AIM :

To implement a login-protected CMS admin panel using Passport.js local strategy, enabling session-based authentication and access control for authorized users only.

ALGORITHM :

Step 1: Initialize an Express.js application and configure session middleware using express-session.

Step 2: Set up passport and use the LocalStrategy to authenticate users.

Step 3: Define user serialization and deserialization functions.

Step 4: Create login and logout routes.

Step 5: Protect the admin route using req.isAuthenticated() to ensure only authenticated users can access it.

Step 6: Serve static HTML files for login, dashboard, and logout pages.

PROGRAM:

APP.JS

```
const express = require('express');

const passport = require('passport');

const LocalStrategy = require('passport-local').Strategy;

const session = require('express-session');

const path =

require('path'); const app

= express();  const PORT =

3001;

// Hardcoded admin user (simulate DB)

const users = [{ id: 1, username: 'admin', password: '1234' }];

// Middleware setup

app.use(express.urlencoded({ extended: false }));

app.use(session({ secret: 'cms', resave: false, saveUninitialized: false }));

app.use(passport.initialize());
```

```
app.use(passport.session());
// Passport Local Strategy
passport.use(new LocalStrategy((username, password, done) => {
    const user = users.find(u => u.username === username && u.password === password);
    return user ? done(null, user) : done(null, false);
}));
// Serialize/Deserialize user
passport.serializeUser((user, done) => done(null, user.id));
passport.deserializeUser((id, done) => {
    const user = users.find(u => u.id === id);
    done(null, user);
});
// Routes
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'login.html'));
});
app.post('/login',
    passport.authenticate('local', { failureRedirect: '/' }),
    (req, res) => {
        res.redirect('/admin');
    }
);
app.get('/admin', (req, res) => {
    if (req.isAuthenticated()) {
        res.sendFile(path.join(__dirname, 'views', 'dashboard.html'));
    } else {
        res.send("+ Unauthorized Access. <a href='/'>Login</a>");
    }
});
```

```
app.get('/logout', (req, res) => {
  req.logout(() => {
    res.sendFile(path.join(__dirname, 'views', 'logout.html'));
  });
});
// Start Server
app.listen(PORT, () => {
  console.log(CMS running at http://localhost:${PORT});
});
```

LOGIN.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Admin Panel Login</title>
</head>
<body style="text-align:center; margin-top:100px;">
  <h2>Admin Panel Login</h2>
  <form method="POST" action="/login">
    <input type="text" name="username" placeholder="admin" required><br><br>
    <input type="password" name="password" placeholder="password" required><br><br>
    <button type="submit">Login</button>
  </form>
</body>
</html>
```

DASHBOARD.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Admin Dashboard</title>
```

```
</head>
<body style=”text-align:center; margin-top:80px;”>
    <h2>Welcome to the Admin Dashboard</h2>
    <p>Only authenticated users can see this.</p>
    <a href=”/logout”>Logout</a>
</body>
</html>
```

LOGOUT.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Logout</title>
</head>
<body style=”text-align:center; margin-top:100px;”>
    <p><strong>You have been logged out.</strong> <a href=”/”>Login again</a></p>
</body>
</html>
```

OUTPUT:

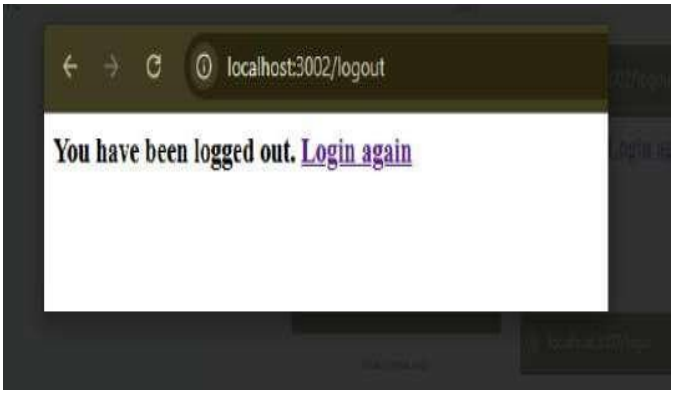
1. Login Page



2. Dashboard Page



3. Logout Page



RESULT:

User authentication for the CMS admin panel was successfully implemented using Passport.js Local Strategy, ensuring access control to protected routes.

EXP NO : 20(b) **USING CONTROLLERS, FILTERS AND CUSTOM DIRECTIVES**

DATE : **ADMIN DASHBOARD FOR STUDENT RECORDS**

AIM :

To develop a student record dashboard using AngularJS that filters student data dynamically and simulates a token-based email verification flow for secure access.

ALGORITHM :

Step 1: Create an HTML page and include AngularJS via CDN.
Step 2: Define the adminApp AngularJS module.
Step 3: Create a controller AdminCtrl with an array of student records.
Step 4: Add a search input using ng-model for filtering.
Step 5: Use AngularJS filter to show matching students.
Step 6: Show a "No matching student" message if nothing matches.
Step 7: Create a studentCard directive to display student info.
Step 8: Style the dashboard using CSS.
Step 9: Run and test the filtering feature in the browser.

PROGRAM :

```
<!DOCTYPE html>
<html ng-app="adminApp">
<head>
  <meta charset="UTF-8">
  <title>Admin Dashboard — Student Records</title>
  <!—AngularJS CDN →
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <!—Google Font and CSS Styling →
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
rel="stylesheet">
  <style>
    body {
      font-family: 'Roboto', sans-serif;
      background: #f4f6f8;
      margin: 0;
      padding: 20px;
    }

    h2 {
      text-align: center;
      color: #2c3e50;
    }

    .dashboard {
      max-width: 600px;
      margin: 30px auto;
      background: white;
      padding: 25px;
      border-radius: 12px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }
```

```
    input[type="text"] {
      width: 100%;
      padding: 12px;
      margin-bottom: 20px;
      border-radius: 6px;
      border: 1px solid #ccc;
      font-size: 16px;
    }

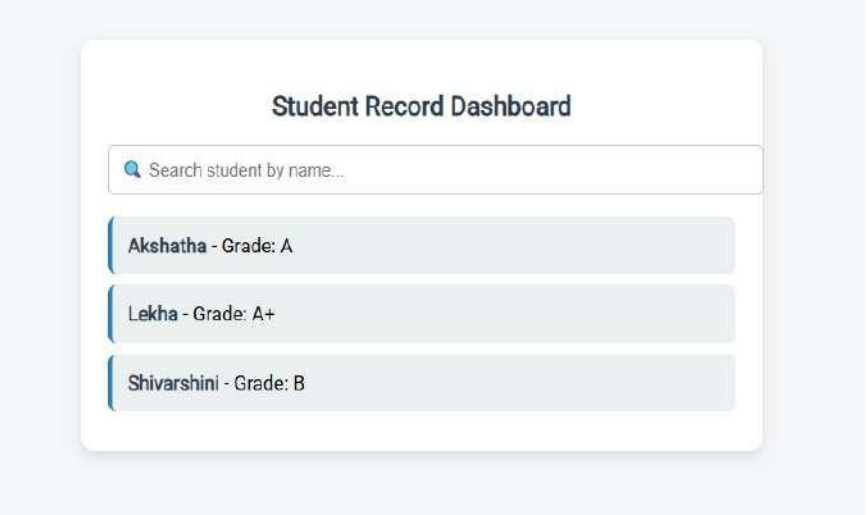
    .student-card {
      padding: 15px;
      margin-bottom: 10px;
      background-color: #ecf0f1;
      border-radius: 8px;
      font-size: 18px;
      border-left: 5px solid #2980b9;
    }
 .student-card strong {
      color: #2c3e50;
    }
   .no-results {
      text-align: center;
      color: #999;
      margin-top: 20px;
    }
  </style>
 </head>
<body ng-controller="AdminCtrl">
  <div class="dashboard">
   <h2>Student Record Dashboard</h2>
  <input type="text" ng-model="search" placeholder=" Search student by name…">
  <div ng-if="(students | filter:search).length === 0" class="no-results">
    No matching student found.
   </div>
 <div ng-repeat="s in students | filter:search">
     <student-card info="s"></student-card>
   </div>
  </div>
<!—AngularJS Script →
  <script>
   angular.module('adminApp', [])
    .controller('AdminCtrl', function($scope) {
      $scope.students = [
        { name: 'Akshatha', grade: 'A' },
        { name: 'Lekha', grade: 'A+' },
        { name: 'Shivarshini', grade: 'B' },

      ];
     })
     .directive('studentCard', function () {
      return {
```

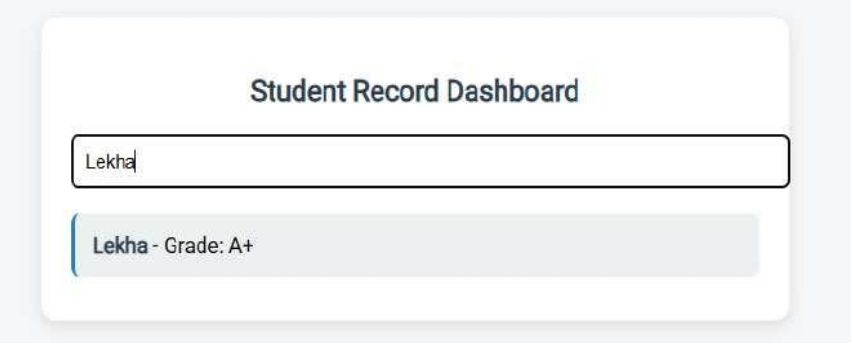
```
        scope: { info: '=' },
        template: '<div class="student-card"><strong>{{info.name}}</strong> - Grade:
{{info.grade}}</div>'
      };
    });
  </script>
</body>
</html>
```

OUTPUT :

1. Dashboard Page



2. Record Page



RESULT :

The system filters and displays student records in real-time based on the entered name using AngularJS.

EXP NO : 21 **JWT AUTHENTICATION**

DATE :

AIM :

To secure RESTful API routes using JSON Web Tokens (JWT) in an Express.js banking application.

ALGORITHM :

Step 1: Setup Express app and enable JSON parsing middleware.

Step 2: Install and configure the jsonwebtoken package.

Step 3: On login (POST /login), generate a token and return it to the client.

Step 4: Create a middleware function to verify the JWT token from the Authorization header.

Step 5: Protect routes (like /account) using the middleware.

Step 6: Return user-specific information only if token is valid.

PROGRAM:

LOGIN.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>JWT Bank Login</title>
</head>
<body style=”text-align:center; margin-top:100px;”>
  <form onsubmit=”login(event)”>
    <h3>Bank Portal Login</h3>
    <input type=”text” id=”username” placeholder=”Username” required><br><br>
    <input type=”password” id=”password” placeholder=”Password” required><br><br>
    <button type=”submit”>Login</button>
  </form>
```

```
    <script>
        async function login€ {
            e.preventDefault();
            const username = document.getElementById(“username”).value;
const res = await fetch(“/login”, {
                method: “POST”,
                headers: { “Content-Type”: “application/json” },
                body: JSON.stringify({ username })
            });
const data = await res.json();
            localStorage.setItem(“token”, data.token);
            window.location.href = “/account”;
        }
    </script>
</body>
</html>
```

ACCOUNT.HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Bank Account</title>
</head>
<body style=”text-align:center; margin-top:80px;”>
    <h2 id=”welcome”>Welcome</h2>
    <p>Account Info Display</p>
 <script>
        async function getAccountInfo() {
            const token = localStorage.getItem(“token”);
```

```
        const res = await fetch("/account", {
            headers: {
                Authorization: "Bearer " + token
            }
        });
if (res.status === 403) {
            alert("Unauthorized. Please login again.");
            localStorage.removeItem("token");
            window.location.href = "/";
        } else {
            const text = await res.text();
            document.getElementById("welcome").innerText = text;
        }
    }
getAccountInfo();
  </script>
</body>
</html>
```

APP.JS

```
const express = require('express');
const jwt =
require('jsonwebtoken'); const
path = require('path');
const app = express();
app.use(express.json());
const SECRET =
'bankjwt';
// Serve login page
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'views', 'login.html'));
```

```
});
```

```
// Serve account page
app.get('/account', (req, res, next) => {
   res.sendFile(path.join(__dirname, 'views', 'account.html'));
});
// POST /login → Return JWT
app.post('/login', (req, res) => {
   const { username } = req.body;
// Generate token valid for 1 hour
   const token = jwt.sign({ user: username }, SECRET, { expiresIn: '1h' });
 res.json({ token });
});
// Middleware to verify JWT
const verify = (req, res, next) => {
   const bearer =
   req.headers['authorization']; if (bearer) {
      const token = bearer.split(' ')[1];
      jwt.verify(token, SECRET, (err, data) => {
         if (err) return res.sendStatus(403);
         req.user = data;
         next();
      });
   } else {
      res.sendStatus(403);
   }
};
// Protected API route
app.get('/account', verify, (req, res) =>
{
   res.send(Welcome ${req.user.user}, Account Info Display);
});
```

```
app.listen(3002, () => {
   console.log('Banking API running at http://localhost:3002');
});
```

OUTPUT :

1. Login Page



2. Dashboard Page



RESULT:

JWT-based login authentication and token-protected route were implemented successfully, matching the banking API use-case. Only authorized users with a valid token can access the account details.

EXP NO : 22(a) **HTML5 CLIENT SIDE FORM VALIDATION FOR**

DATE : **USER REGISTRATION**

AIM :

To ensure that user input in a registration form is valid and meets the required format using built-in HTML5 form validation attributes without writing any JavaScript code.

ALGORITHM :

Step 1: Create a user registration form with input fields like name, email, password, etc.
Step 2: Add HTML5 validation attributes such as required, type="email", pattern, minlength, etc., to each field.
Step 3: When the user submits the form, the browser automatically validates the inputs.
Step 4: If any field is invalid, the browser blocks submission and shows built-in error messages.
Step 5: If all inputs are valid, the form is successfully submitted to the server.

PROGRAM :

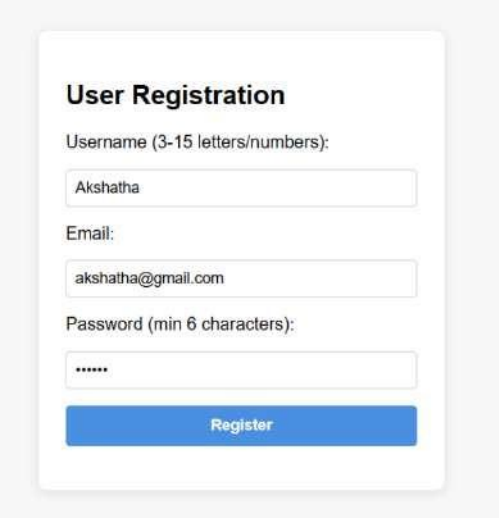
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Simple Form Validation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f7f7f7;
      display: flex;
      justify-content: center;
      padding: 40px 20px;
    }
    form {
      background: white;
```

```
  padding: 25px;
  border-radius: 8px;
  max-width: 320px;
  width: 100%;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
label, input {
  display: block;
  width: 100%;
  margin-bottom: 15px;
}
input {
  padding: 8px;
  font-size: 14px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
}
input[type="submit"] {
  background-color: #4a90e2;
  color: white;
  border: none;
  cursor: pointer;
  font-weight: bold;
  padding: 10px;
}
input[type="submit"]:hover {
  background-color: #357abd;
}
```

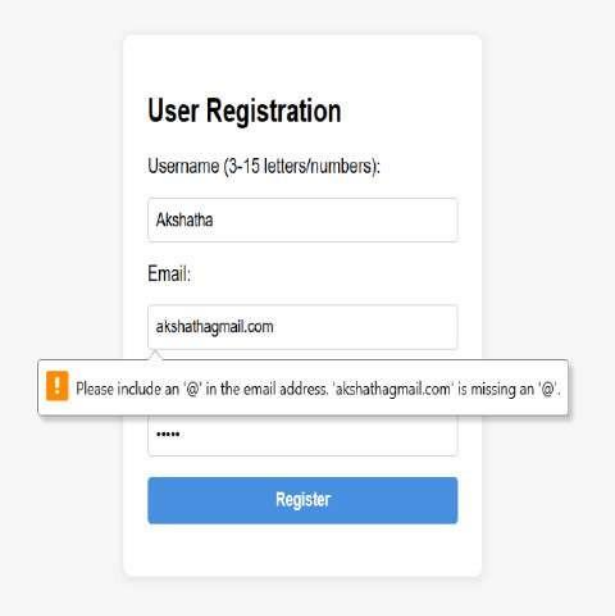
```
  </style>
</head>
<body>
  <form id="userForm">
    <h2>User Registration</h2>
    <label for="username">Username (3-15 letters/numbers):</label>
    <input type="text" id="username" name="username" pattern="[A-Za-z0-9]{3,15}"
required />
<label for="email">Email:</label>
    <input type="email" id="email" name="email" required />
<label for="password">Password (min 6 characters):</label>
    <input type="password" id="password" name="password" minlength="6" required />
<input type="submit" value="Register" />
  </form>
<script>
    document.getElementById('userForm').addEventListener('submit', function(e) {
      if (!this.checkValidity()) {
        e.preventDefault();
        alert('Please fill out the form correctly.');
      }
    });
  </script>
</body>
</html>
```

OUTPUT :

1. Register Page



2. Invalid Register Page



RESULT :

User is prevented from submitting incomplete or incorrectly formatted data, ensuring basic input correctness using browser-based validation.

EXP NO : 22(b) **CLIENT SIDE FORM VALIDATION USING JAVASCRIPT**

DATE :

AIM :

To manually validate user input on the client-side using JavaScript before the form is submitted.

ALGORITHM :

Step 1: Create a form with input fields for user details (e.g., name, email, password).
Step 2: Write a JavaScript function to be triggered on form submission.
Step 3: Use JavaScript to access and read values from form fields.
Step 4: Apply validation checks (e.g., required fields, proper email format, password length, matching passwords).
Step 5: If validation fails, prevent form submission and display error messages.
Step 6: If all validations pass, allow the form to submit to the server.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Client-Side Form Validation with JavaScript</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f7f7f7;
      display: flex;
      justify-content: center;
      padding: 40px 20px;
    }
    form {
      background: white;
```

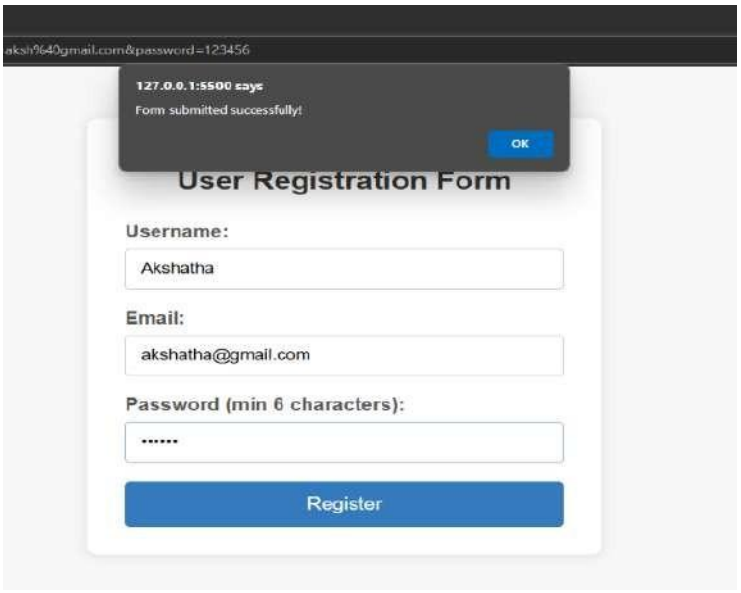
```
  padding: 25px 30px;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  max-width: 350px;
  width: 100%;
}
h2 {
  text-align: center;
  margin-bottom: 25px;
  color: #333;
}
label {
  font-weight: bold;
  display: block;
  margin-bottom: 6px;
  color: #555;
}
input[type="text"],
input[type="email"],
input[type="password"] {
width: 100%;
  padding: 10px 12px;
  margin-bottom: 18px;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 14px;
  box-sizing: border-box;
  transition: border-color 0.3s ease;
}
```

```
    input[type="text"]:focus,
    input[type="email"]:focus,
    input[type="password"]:focus {
    border-color: #4a90e2;
      outline: none;
    }
    input[type="submit"] {
      width: 100%;
      padding: 12px;
      background-color: #4a90e2;
      color: white;
      border: none;
      border-radius: 4px;
      font-size: 16px;
      cursor: pointer;
      transition: background-color 0.3s ease;
    }
    input[type="submit"]:hover {
      background-color: #357abd;
    }
  </style>
</head>
<body>
  <form id="userForm" novalidate>
    <h2>User Registration Form</h2>
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" placeholder="Enter username" />
<label for="email">Email:</label>
    <input type="email" id="email" name="email" placeholder="example@example.com" />
```

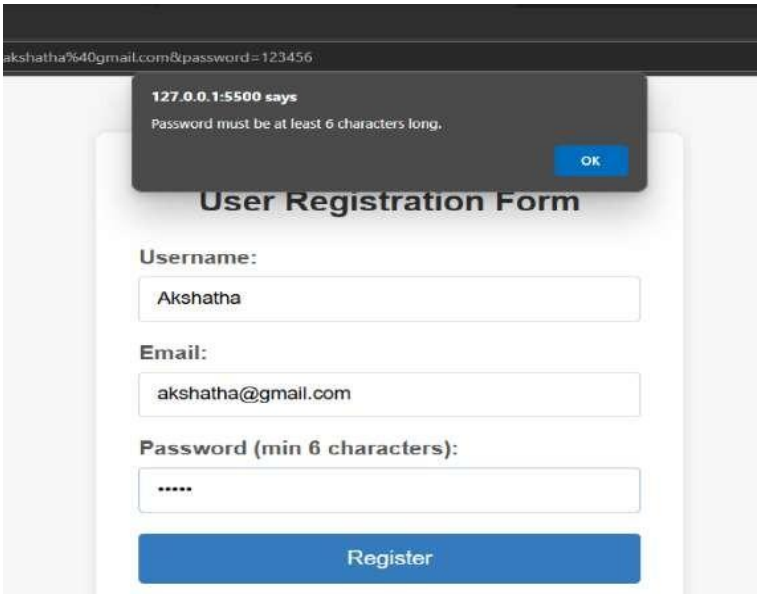
```
<label for="password">Password (min 6 characters):</label>
    <input type="password" id="password" name="password" placeholder="Enter password"
/>
<input type="submit" value="Register" />
  </form>
<script>
    const form = document.getElementById('userForm');
    form.addEventListener('submit', e => {
    e.preventDefault();
      const username = form.username.value.trim();
      const email = form.email.value.trim();
      const password = form.password.value;
      const usernameRegex = /^[A-Za-z0-9]{3,15}$/;
      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 if (!username) return alert('Username is required.'), form.username.focus();
      if (!usernameRegex.test(username)) return alert('Username must be 3-15 characters long
and contain only letters and numbers.'), form.username.focus();
      if (!email) return alert('Email is required.'), form.email.focus();
      if (!emailRegex.test(email)) return alert('Please enter a valid email address.'),
form.email.focus();
      if (!password) return alert('Password is required.'), form.password.focus();
      if (password.length < 6) return alert('Password must be at least 6 characters long.'),
form.password.focus();
alert('Form submitted successfully!');
      form.submit();
    });
  </script>
</body>
</html>
```

OUTPUT :

1. **Register Page**



2. Invalid Register Page



RESULT:

Provides a customizable and interactive validation experience to ensure proper input before sending data to the server.

EXP NO : 23(a) **SERVER SIDE VALIDATION USING EXPRESS.JS AND JOI**

DATE :

AIM :

To validate and sanitize user inputs on the server using the Joi library in an Express.js application.

ALGORITHM :

Step 1: Install and import the Joi library in the Express project.
Step 2: Define a validation schema using Joi for the expected input fields.
Step 3: In the route handler, extract user data from the request body.
Step 4: Use the Joi schema to validate the extracted data.
Step 5: If validation fails, return an error response to the client with details.
Step 6: If validation succeeds, proceed with processing the data (e.g., save to database).

PROGRAM:

```
const express = require('express');

const Joi = require('joi');

const app = express();

app.use(express.json());

app.get('/', (req, res) => {

res.send('Server is

working');

});

const schema = Joi.object({

  name: Joi.string().min(3).required(),

  email: Joi.string().email().required(),

  password: Joi.string().min(6).required()

});

app.post('/register', (req, res) => {

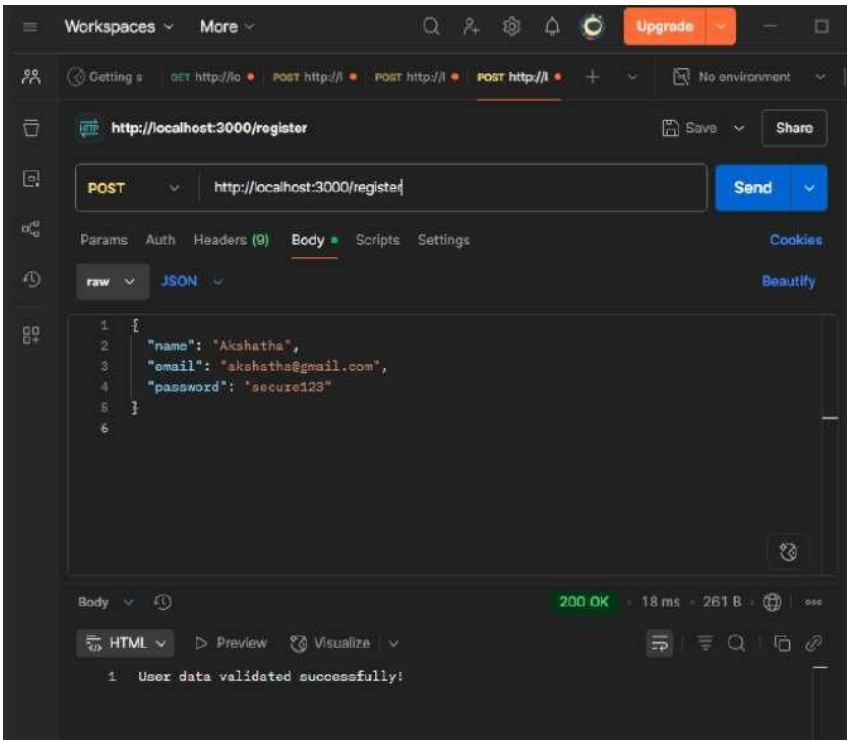
  const { error } = schema.validate(req.body);


  if (error) {
```

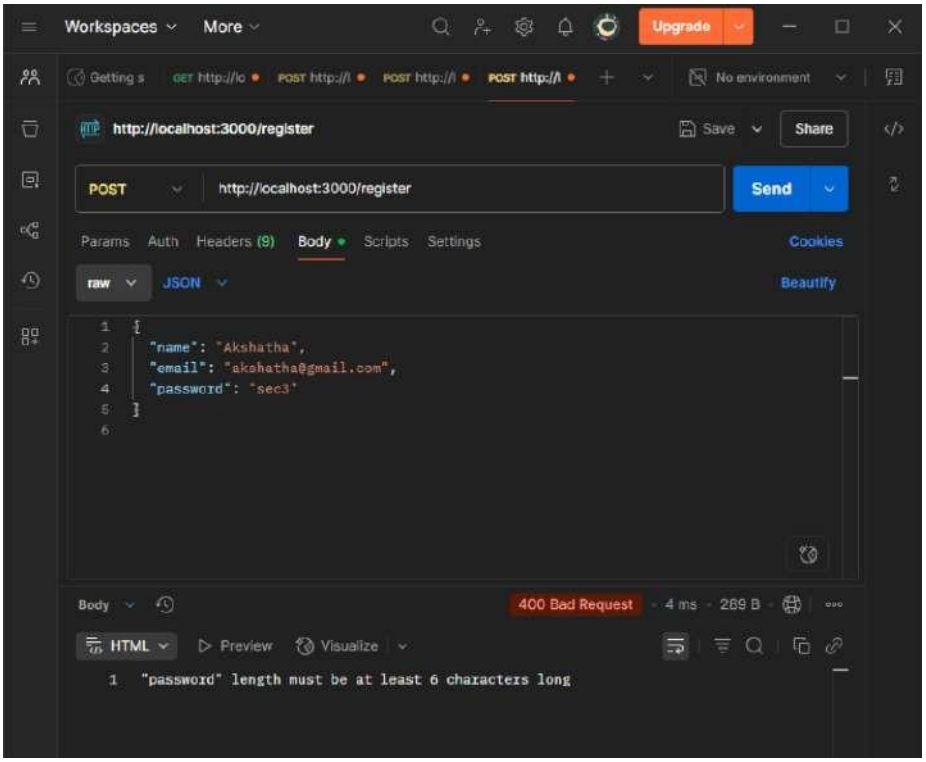
```
    return res.status(400).send(error.details[0].message);
  }
res.send('User data validated successfully!');
});
app.listen(3000, () => {
  console.log(' Server is running on port 3000');
});
```

OUTPUT :

1. Validation



2. Invalidation



RESULT:

Server accepts and processes only correctly structured data, protecting against invalid and malicious inputs.

EXP NO : 23(b) **SERVER SIDE VALIDATION USING EXPRESS-VALIDATOR**

DATE :

AIM :

To perform structured server-side input validation in an Express application using the express-validator middleware.

ALGORITHM :

Step 1: Install and configure the express-validator package in the Express project.
Step 2: Define validation and sanitization rules for each input field using express-validator functions.
Step 3: Attach these rules as middleware to the route handling form submission.
Step 4: On form submission, check the request for validation errors.
Step 5: If errors exist, return a response with the list of validation messages.
Step 6: If no errors are found, continue with processing the valid data.

PROGRAM:

```
const express = require('express');

const { body, validationResult } = require('express-validator');

const app = express();

app.use(express.json());

app.get('/', (req, res) => {

  res.send('Welcome to the homepage! Go to /login to login.');

});

app.get('/login', (req, res) => {

  res.send('This is the login page. Please send a POST request with email and password in JSON.');

});

app.post(

  '/login',

  body('email').isEmail().withMessage('Enter a valid email address'),

  body('password')

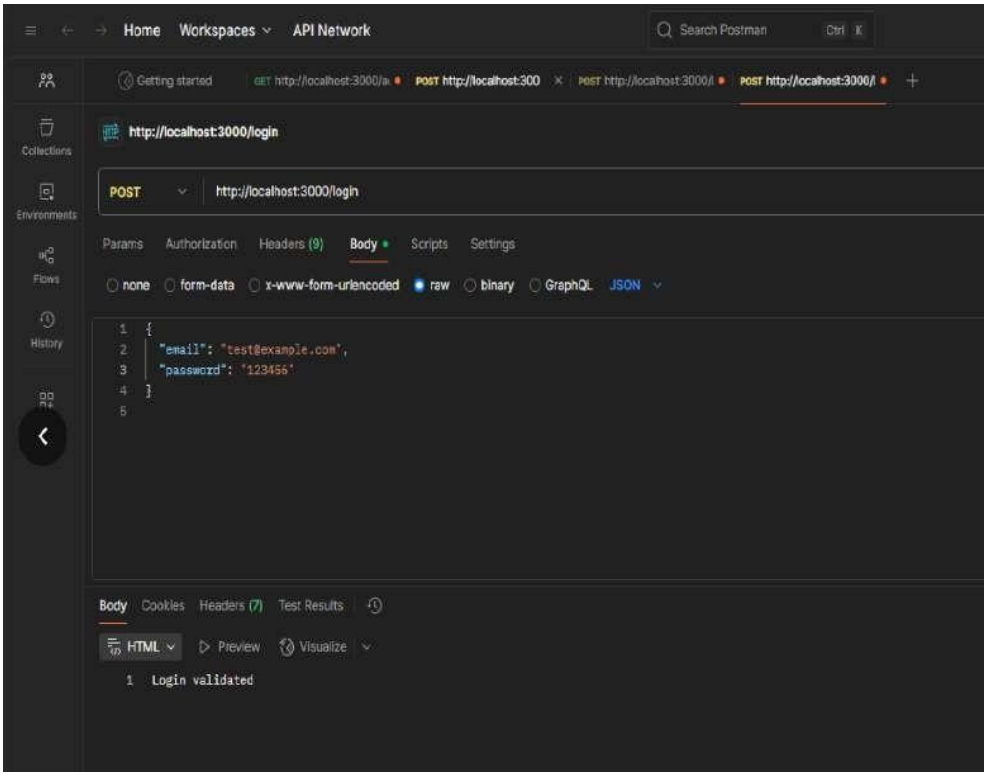
    .isLength({ min: 6 })

    .withMessage('Password must be at least 6 characters long'),
```

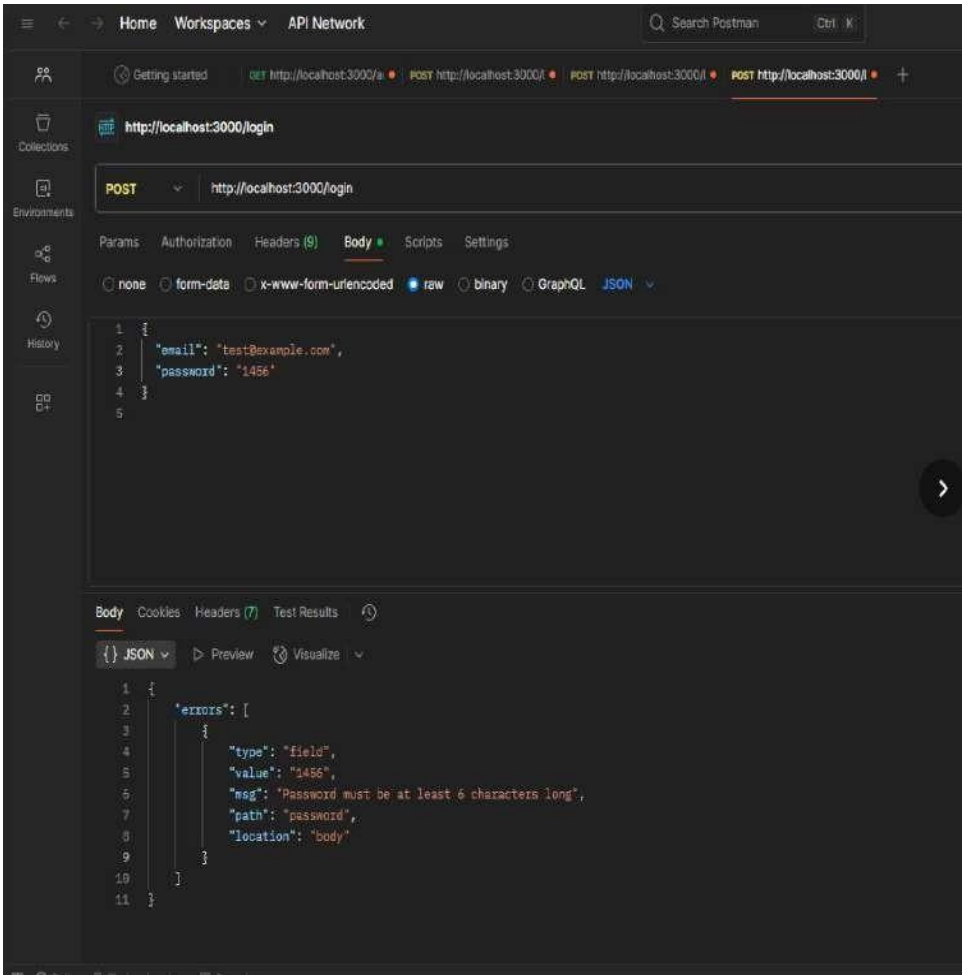
```
(req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    res.send('Login validated');
  }
);
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

OUTPUT :

1. Validation



2. Invalidation



RESULT:

Ensures input data is validated and sanitized server-side, improving data integrity and application security.

EXP NO : 24(a)

DATE :

TWO-WAY DATA BINDING IN ANGULARJS – REAL-TIME STUDENT PROFILE PREVIEW

AIM :

To create a real-time student profile preview interface using AngularJS two-way data binding.

ALGORITHM :

Step 1: Start the process.
Step 2: Create an HTML form with input fields using ng-model for student details.
Step 3: Bind the input fields to a preview section using AngularJS {{ }} expressions.
Step 4: As the user types, changes are reflected instantly in the preview section.
Step 5: End the process.

PROGRAM:

```
<!DOCTYPE html>
<html ng-app=”studentApp”>
<head>
  <meta charset=”UTF-8”>
  <title>Student Profile Preview</title>
  <script src=”https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js”></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
    }
    input {
      padding: 6px;
      margin: 5px 0;
      width: 300px;
    }
    .preview {
      margin-top: 20px;
      padding: 10px;
      background: #f2f2f2;
      border-left: 4px solid #007BFF;
    }
  </style>
</head>
<body>
<div ng-controller=”StudentCtrl”>
  <h2>Student Registration Form</h2>
  <label>Name:</label><br>
```

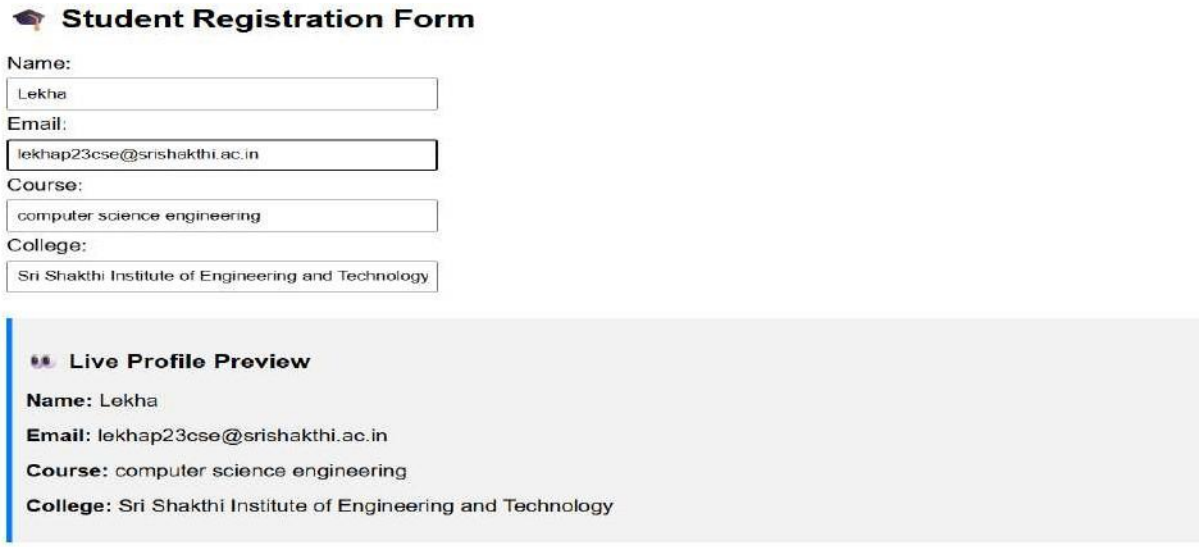
```
  <input type=”text” ng-model=”student.name”><br>
  <label>Email:</label><br>
  <input type=”email” ng-model=”student.email”><br>
  <label>Course:</label><br>
  <input type=”text” ng-model=”student.course”><br>
  <label>College:</label><br>
  <input type=”text” ng-model=”student.college”><br>
  <div class=”preview”>
    <h3>Live Profile Preview</h3>
    <p><strong>Name:</strong> {{student.name}}</p>
    <p><strong>Email:</strong> {{student.email}}</p>
    <p><strong>Course:</strong> {{student.course}}</p>
    <p><strong>College:</strong> {{student.college}}</p>
  </div>
</div>
<script>
  angular.module(‘studentApp’, [])
    .controller(‘StudentCtrl’, function($scope) {
      $scope.student = {
        name: ‘’,
        email: ‘’,
        course: ‘’,
        college: ‘’
      };
    });
</script>
</body>
</html>
```

OUTPUT:

1. Registration Page



2. Profile page



RESULT:

The real-time student profile preview was successfully implemented using AngularJS. Changes made in the form fields were instantly reflected in the preview section through two-way data binding.

EXP NO : 24(b) **FETCHING AND DISPLAYING DATA FROM MY SQL**

DATE : **REAL TIME STUDENT ATTENDANCE VIEWER**

AIM :

To display student attendance records from a MySQL database in real-time using Node.js and AngularJS.

.

ALGORITHM :

Step 1: Store attendance data in a MySQL database.

Step 2: Create a Node.js API to fetch attendance records.

Step 3: Use AngularJS to call the API using $http.

Step 4: Display the attendance data in a table on the webpage.

PROGRAM:

```
 const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');
const app = express();
app.use(cors());
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'college'
});
db.connect((err) => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
app.get('/attendance', (req, res) => {
  db.query('SELECT * FROM attendance', (err, result) => {
```

```
    if (err) throw err;
    res.json(result);
  });
});
app.listen(3000, () => {
console.log('Server running on http://localhost:3000');
});
CREATE DATABASE college;
USE college;
CREATE TABLE attendance (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student VARCHAR(100),
  date DATE,
  status VARCHAR(20)
);
INSERT INTO attendance (student, date, status) VALUES
('Akshatha', '2025-08-05', 'Present'),
('Abc', '2025-08-05', 'Absent'),
('Xyz', '2025-08-05', 'Present');
<!DOCTYPE html>
<html lang="en" ng-app="attApp">
<head>
  <meta charset="UTF-8">
  <title>Attendance Viewer</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    body {
      background-color: #f4f4f4;
      padding-top: 40px;
      font-family: Arial, sans-serif;
    }
    .container {
      max-width: 700px;
      background-color: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    h2 {
      margin-bottom: 20px;
      text-align: center;
      color: #333;
    }
  </style>
</head>
<body ng-controller="AttCtrl">
```

```
<div class="container">
  <h2>Student Attendance Records</h2>
  <table class="table table-bordered">
    <thead class="table-light">
      <tr>
        <th>#</th>
        <th>Student</th>
        <th>Date</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr ng-repeat="a in attendance track by $index">
        <td>{{$index + 1}}</td>
        <td>{{a.student}}</td>
        <td>{{a.date}}</td>
        <td>{{a.status}}</td>
      </tr>
    </tbody>
  </table>
</div>
<script>
  angular.module('attApp', [])
    .controller('AttCtrl', function($scope, $http) {
      $http.get('http://localhost:3000/attendance')
        .then(function(res) {
          $scope.attendance = res.data;
        });
    });
</script>
</body>
</html>
```

OUTPUT:



RESULT:

Student attendance is displayed in real-time from the MySQL database on the web page.

EXP NO : 25(a) **CUSTOM ERROR HANDLING MIDDLEWARE IN EXPRESS**

DATE :

AIM :

To manage application errors in a centralized way using custom error-handling middleware in Express.js.

ALGORITHM :

Step 1: Define a middleware function that accepts four parameters: err, req, res, and next.
Step 2: After all route definitions, add this error-handling middleware to the Express app.
Step 3: When an error occurs in any route, pass it using next(error).
Step 4: Inside the middleware, log or format the error for debugging or user display.
Step 5: Send a structured and consistent error response to the client (e.g., status code and message).
Step 6: Optionally, customize responses for different types of errors (validation, not found, server errors, etc.).

PROGRAM:

```
const express = require('express');

const app = express();

app.use(express.json());

const users = [

  { id: 1, name: 'Alice' },

  { id: 2, name: 'Bob' }

];

app.get('/', (req, res) => {

  res.send('Welcome to the Express Error Handling Example!');

});

app.get('/users/:id', (req, res, next) => {

const id = parseInt(req.params.id, 10);

const user = users.find(u => u.id === id);

if (!user) {

    const err = new Error('User not found');
```

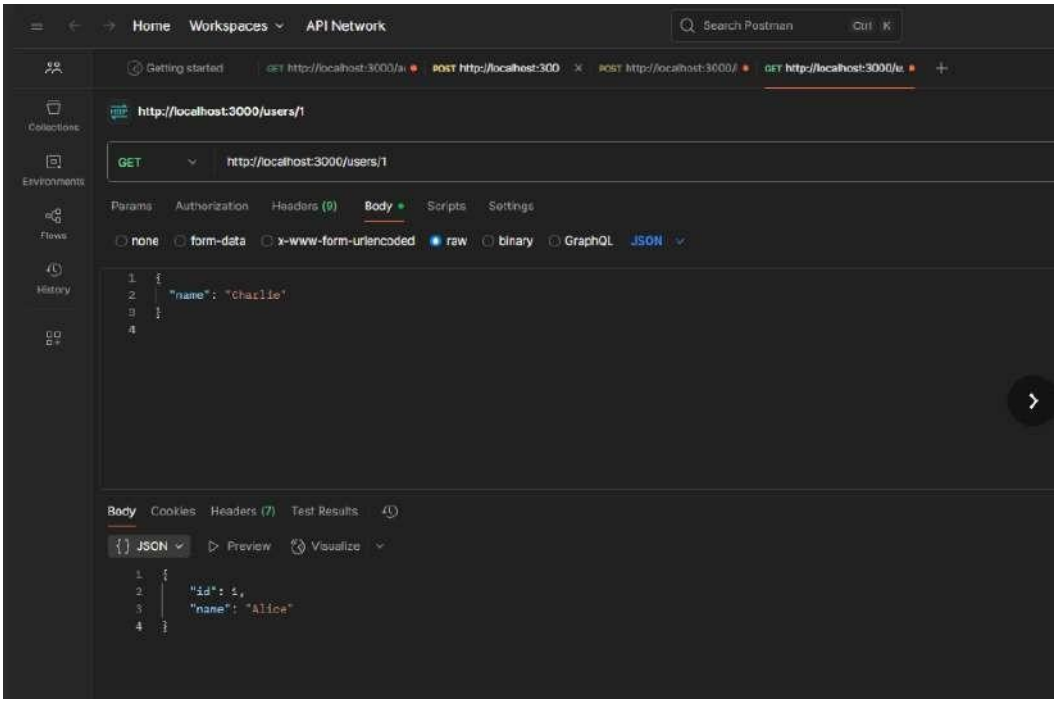
```
    err.status = 404;
    return next(err);
  }

res.json(user);
});
app.post('/users', (req, res, next) =>
  { try {
    const { name } = req.body;
    if (!name) {
      throw new Error('Name is required');
    }
const newUser = {
      id: users.length + 1,
      name
    };
users.push(newUser);
    res.status(201).json(newUser);
  } catch (error) {
    next(error);
  }
});
app.use((err, req, res, next) => {
console.error('Error:', err.message);
res.status(err.status || 500).json({
    error: {
      message: err.message || 'Internal Server Error',
      status: err.status || 500
    }
```

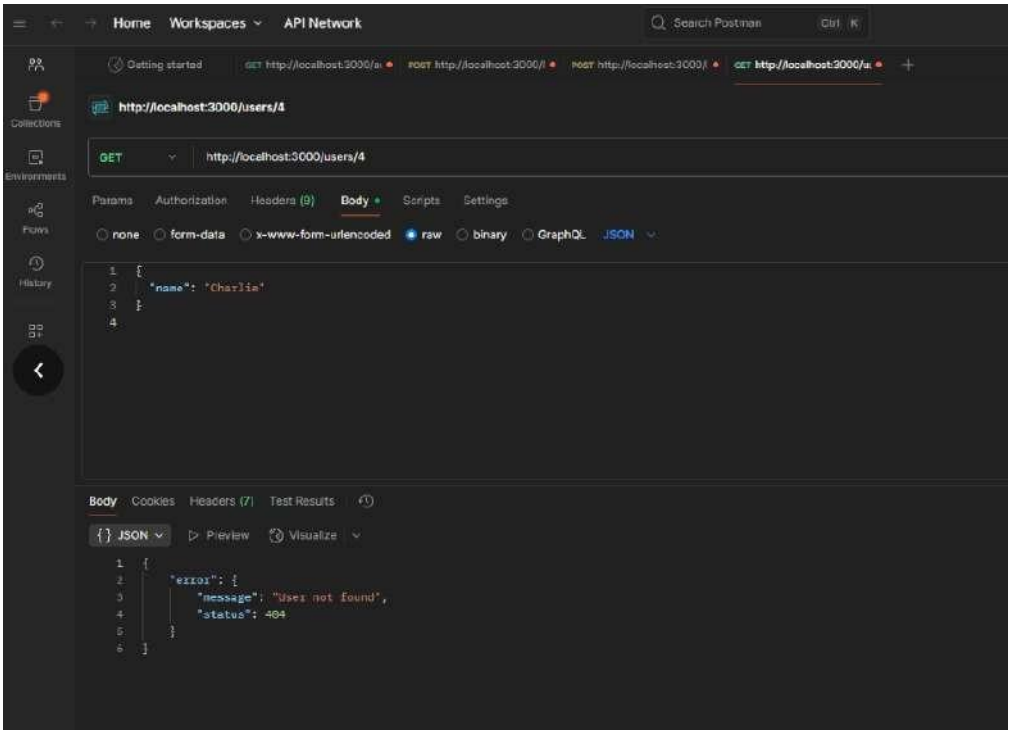
```
  });
});
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

OUTPUT :

1. Validation



2. Invalidation



RESULT:

Provides a clean and consistent way to handle all application errors, improving maintainability and user feedback.

EXP NO : 25(b) **ANGULAR JS FORM VALIDATION COLLEGE REGISTRATION**

DATE **:** **FORM WITH REAL TIME ERROR HANDLING**

AIM :

To design a secure college registration form using AngularJS with input validation, simulating a secure login flow that includes email verification and JWT (JSON Web Token) authentication.

ALGORITHM :

Step 1: Create an AngularJS app named collegeApp.
Step 2: Add a controller FormCtrl to manage form data.
Step 3: Design the form with username and password fields.
Step 4: Use ng-model, required, and ng-minlength for validation.
Step 5: Show error messages with ng-show on invalid inputs.
Step 6: Disable the "Register" button until the form is valid.
Step 7: Style the form using CSS for better UX.
Step 8: (In full flow) Send a JWT link to email for verification.
Step 9: Verify token on click to activate the account.
Step 10: Test the form in the browser.

PROGRAM:

```
<!DOCTYPE html>

<html ng-app="collegeApp">

<head>

  <meta charset="UTF-8">

  <title>College Registration Form</title>

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

  <style>

    body {

      font-family: 'Segoe UI', sans-

      serif; background: #f2f6fc;

      padding: 40px;

      font-size: 18px; /* Base font size increased */

    }

.form-box {

      max-width: 500px;
```

```
        margin: auto;
        background: white;
        padding: 40px;
        border-radius: 12px;
        box-shadow: 0 4px 15px rgba(0,0,0,0.1);
      }
    h2 {
        text-align: center;
        margin-bottom: 30px;
        color: #2c3e50;
        font-size: 28px;
      }
     label {
        font-weight: 600;
        margin-top: 20px;
        display: block;
        color: #34495e;
        font-size: 18px;
      }
    input {
        width: 100%;
        padding: 14px 16px;
        margin-top: 8px;
        border: 1px solid #ccc;
        border-radius: 8px;
        font-size: 17px;
        box-sizing: border-box;
        transition: border-color 0.3s;
      }
     input:focus {
```

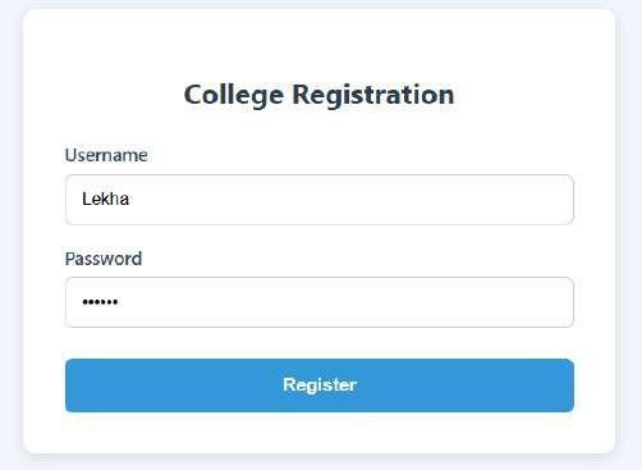
```
      border-color: #3498db;
      outline: none;
    }
  .error-msg {
      color: red;
      font-size: 15px;
      margin-top: 6px;
    }
 button {
      margin-top: 30px;
      width: 100%;
      padding: 16px;
      background-color: #3498db;
      color: white;
      font-size: 18px;
      font-weight: bold;
      border: none;
      border-radius: 8px;
      cursor: pointer;
      transition: background-color 0.3s;
    }
  button:hover {
      background-color: #2980b9;
    }
 button:disabled {
      background-color: #ccc;
      cursor: not-allowed;
    }
   </style>
 </head>
```

```
<body ng-controller=”FormCtrl”>
  <div class=”form-box”>
    <h2>College Registration</h2>
<form name=”regForm” novalidate>
<!—Username →
      <label for=”username”>Username</label>
      <input type=”text” name=”username” ng-model=”user.username” required
placeholder=”Enter your username”>
      <div class=”error-msg” ng-show=”regForm.username.$touched &&
regForm.username.$invalid”>
        * Username is required.
      </div>
<!—Password →
      <label for=”pass”>Password</label>
      <input type=”password” name=”pass” ng-model=”user.pass” required ng-minlength=”6”
placeholder=”Minimum 6 characters”>
      <div class=”error-msg” ng-show=”regForm.pass.$touched &&
regForm.pass.$error.required”>
        * Password is required.
      </div>
      <div class=”error-msg” ng-show=”regForm.pass.$touched &&
regForm.pass.$error.minlength”>
        * Password must be at least 6 characters.
      </div>
<!—Submit →
      <button type=”submit” ng-disabled=”regForm.$invalid”>Register</button>
</form>
  </div>
 <script>
    angular.module(‘collegeApp’, [])
      .controller(‘FormCtrl’, function($scope) {
        $scope.user = {};
```

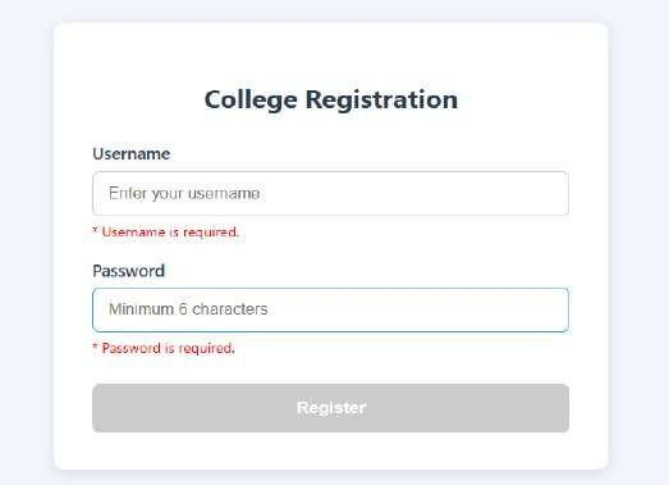
```
    });
  </script>
</body>
</html>
```

OUTPUT:

1. Registration Page



2. Invalid Registration Page



RESULT:

A responsive and secure AngularJS registration form is created with real-time validation to simulate email verification and JWT-based login flow

EXP NO : 26 **INTEGRATION WITH OPEN WEATHER API**

DATE : **(ADMIN DASHBOARD USE CASE)**

AIM :

To fetch and display real-time weather data in the admin dashboard by integrating the OpenWeather API.

ALGORITHM :

Step 1: Register and obtain an API key from https://openweathermap.org/.

Step 2: Create a frontend input for the admin to enter a city name.

Step 3: Set up a backend route (e.g., /get-weather) to call OpenWeather API.

Step 4: Make an HTTP request (using axios or fetch) to the OpenWeather API with the given city and API key.

Step 5: Parse the response to extract necessary weather details (temperature, humidity, condition).

Step 6: Send the weather data from backend to frontend.

Step 7: Display the weather details in the admin dashboard with proper UI design.

PROGRAM :

```
const express = require('express');
const path = require('path');
const fetch = require('node-fetch'); // install with `npm install node-fetch@2`
const app = express();
const PORT = 3000;
const API_KEY = '537ae97715ed14df460ce9ceab69c431';
app.use(express.static('public'));
app.use(express.urlencoded({ extended: true }));
app.post('/getweather', async (req, res) => {
  const city = req.body.city;
  if (!city) {
```

```
        return res.json({ success: false, error: 'City name is required' });
    }
    try {
        const url =
`https://api.openweathermap.org/data/2.5/weather?q=${encodeURIComponent(city)}&appid
=${API_KEY}&units=metric`;
        const response = await fetch(url);
        const data = await response.json();
    if (data.cod !== 200) {
            return res.json({ success: false, error: data.message || 'City not found' });
        }
const weather = {
            city: data.name,
            temp: data.main.temp,
            description: data.weather[0].description,
            icon: data.weather[0].icon
        };
  res.json({ success: true, weather });
    } catch (err) {
        res.json({ success: false, error: 'Failed to fetch weather data' });
    }
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<title>Weather Dashboard</title>
```

```
<style>
  body {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
    display: flex;
    height: 100vh;
    justify-content: center;
    align-items: center;
    margin: 0;
  }
  .container {
    background: white;
    padding: 25px 30px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    width: 320px;
    text-align: center;
  }
  h1 {
    margin-bottom: 20px;
    font-weight: normal;
  }
  input[type=”text”] {
    width: calc(100% - 22px);
    padding: 10px;
    font-size: 16px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
```

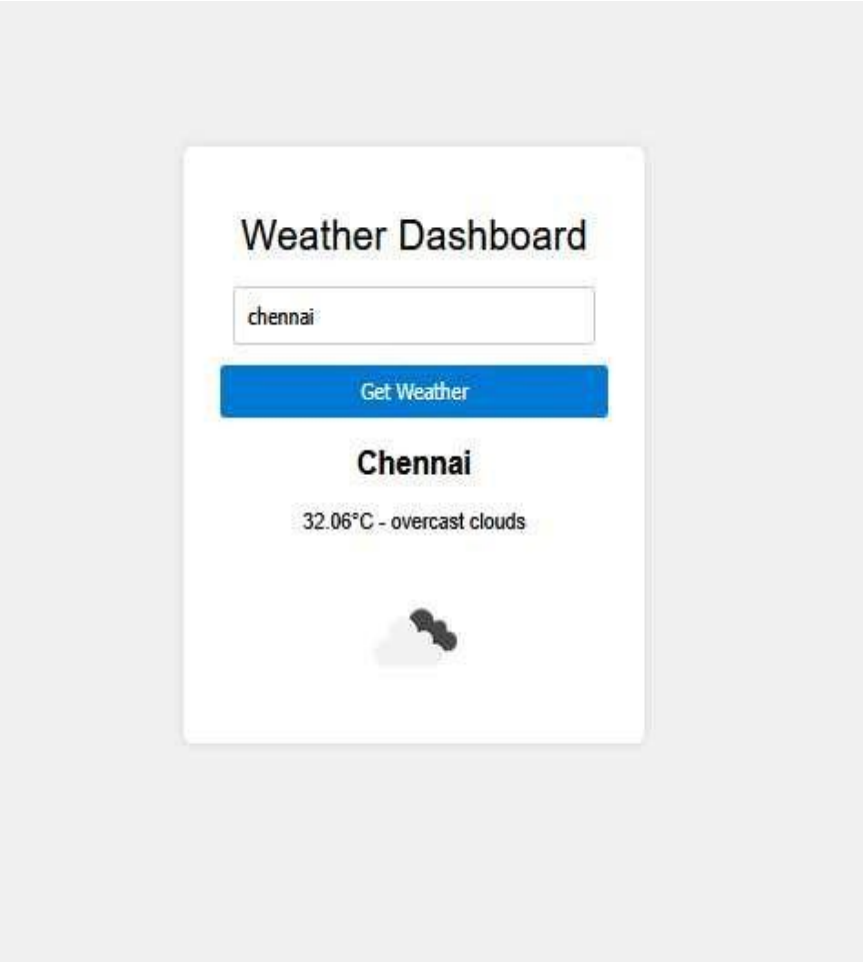
```
    box-sizing: border-box;
  }
  button {
    width: 100%;
    padding: 10px;
    background-color: #0078d4;
    border: none;
    color: white;
    font-size: 16px;
    cursor: pointer;
    border-radius: 4px;
  }
  button:hover {
    background-color: #005a9e;
  }
  #result {
    margin-top: 20px;
  }
  #result img {
    margin-top: 10px;
  }
  p.error {
    color: red;
    font-weight: bold;
  }
</style>
</head>
<body>
```

```
<div class=”container”>
  <h1>Weather Dashboard</h1>
  <input type=”text” id=”city” placeholder=”Enter city name” />
  <button onclick=”getWeather()”>Get Weather</button>
  <div id=”result”></div>
</div>
<script>
async function getWeather() {
  const city = document.getElementById(‘city’).value.trim();
  const resultDiv = document.getElementById(‘result’);
  if (!city) {
    resultDiv.innerHTML = ‘<p class=”error”>Please enter a city name.</p>’;
    return;
  }
  resultDiv.textContent = ‘Loading…’;
 try {
    const response = await fetch(‘/getweather’, {
      method: ‘POST’,
      headers: { ‘Content-Type’: ‘application/x-www-form-urlencoded’ },
      body: `city=${encodeURIComponent(city)}`
    });
const data = await response.json();
    if (data.success) {
    resultDiv.innerHTML = `
        <h2>${data.weather.city}</h2>
        <p>${data.weather.temp}°C - ${data.weather.description}</p>
        <img src=”http://openweathermap.org/img/wn/${data.weather.icon}@2x.png”
alt=”Weather Icon” />
      `;
```

```
    } else {
      resultDiv.innerHTML = `<p class="error">${data.error || 'Error fetching weather data.'}</p>`;
    }
  } catch (err) {
    resultDiv.innerHTML = '<p class="error">Network error. Please try again later.</p>';
  }
}
</script>
</body>
</html>
```

OUTPUT :





RESULT :

The admin dashboard successfully displays current weather information based on the city entered, using live data from the OpenWeather API**.**

EXP NO : 27(a) **SENDING EMAIL WITH NODEMAILER**

DATE : **(USER REGISTRATION CONFIRMATION)**

AIM:

To send an email confirmation to the user after successful registration using Nodemailer.

ALGORITHM :

Step 1: Install and configure nodemailer in your Node.js backend.

Step 2: Set up SMTP transporter with your email service (e.g., Gmail).

Step 3: Create an email template for confirmation with a subject and message body.

Step 4: When the user registers, trigger the email sending function.

Step 5: Use the transporter to send the email with user's email ID.

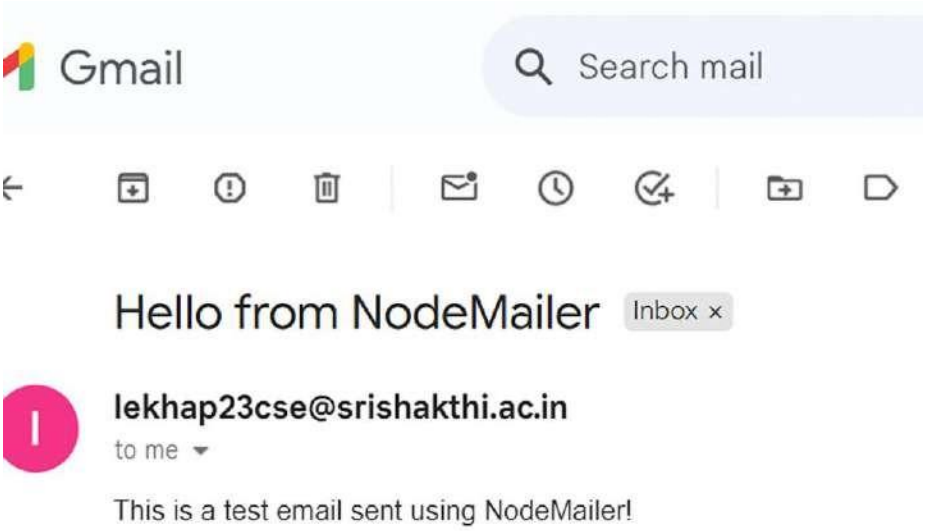
Step 6: Handle success or error responses and log them for debugging.

PROGRAM

```
const nodemailer = require('nodemailer');
let transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'akshathaprabakaran23cse@srishakthi.ac.in',
    pass: 'vlkd asbq dydf zipk'
  }
});
let mailOptions = {
  from: 'akshathaprabakaran23cse@srishakthi.ac.in',
  to: 'lekhap23cse@srishakthi.ac.in',
  subject: 'Hello from NodeMailer',
  text: 'This is a test email sent using NodeMailer!'
};
```

```
transporter.sendMail(mailOptions, function(error, info){
  if (error) {
    return console.log(error);
  }
  console.log('Message sent: ' + info.response);
});
```

OUTPUT:



RESULT :

Upon registration, the user receives a confirmation email in their inbox, indicating successful signup and providing further instructions if needed

EXP NO : 27(b) **ADDING ANGULARJS ANIMATIONS – HR NOTIFICATION**

DATE : **POPUPS IN AN EMPLOYEE PORTAL**

AIM :

To implement and display HR notifications with smooth entry and exit animations using AngularJS and ngAnimate.

ALGORITHM :

Step 1: Create an AngularJS module with ngAnimate as a dependency.
Step 2: Add a controller with a $scope variable to control notification visibility.
Step 3: Use ng-if in HTML to display the notification message when the variable is true.
Step 4: Define CSS animation rules for .ng-enter and .ng-leave classes.
Step 5: Toggle the $scope variable to animate showing and hiding the popup.

PROGRAM:

index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="employeePortal">
<head>
  <meta charset="UTF-8">
  <title>Employee Portal Dashboard</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/angular-animate/1.8.2/angular-animate.min.js"></script>
  <style>
    body {
    margin: 0;
      font-family: 'Segoe UI', Tahoma, sans-serif;
      background-color: #f4f6f8;}
    header {
      background-color: #1e3a8a;
      color: white;
      padding: 15px 30px;
      font-size: 24px;}
    .dashboard {
      padding: 40px;}
    .hr-popup {
      background-color: #fff9db;
      border-left: 6px solid #ffcc00;
      box-shadow: 0 5px 10px rgba(0,0,0,0.15);
      padding: 20px 25px;
      max-width: 500px;
      position: relative;
```

```
      border-radius: 10px;
      font-size: 16px;
      margin: 0 auto; }
    .hr-popup h3 {
      margin-top: 0;
      font-size: 20px;}
    .hr-popup p {
      margin: 8px 0;}
    .hr-popup .close-btn {
      position: absolute;
      top: 10px;
      right: 15px;
      font-size: 20px;
      border: none;
      background: none;
      color: #666;
      cursor: pointer;}
    .hr-popup .close-btn:hover {
      color: #000;}
    .view-link {
      display: inline-block;
      margin-top: 10px;
      text-decoration: none;
      color: #1e3a8a;
      font-weight: bold;}
    .view-link:hover {
      text-decoration: underline;}
    .hr-popup.ng-enter {
      opacity: 0;
      transform: translateY(-20px);}
    .hr-popup.ng-enter-active {
      opacity: 1;
      transform: translateY(0);
      transition: all 0.5s ease;}
    .hr-popup.ng-leave {
      opacity: 1;
      transform: translateY(0);}
  </style>
</head>
<body ng-controller="PortalController">
  <header>Employee Self-Service Portal</header>
  <div class="dashboard">
    <div class="hr-popup" ng-if="showPopup">
      <button class="close-btn" ng-click="closePopup()">&times;</button>
```

```
    <h3>HR Notification</h3>
    <p><strong>Date:</strong> 8 August 2025</p>
    <p><strong>Subject:</strong> Payroll Credit Update</p>
    <p>This month's salary will be credited on 28th August due to scheduled bank maintenance.</p>
    <a href="#" class="view-link">View All Notices</a>
  </div><div style="margin-top: 60px;">
    <h2>Welcome, {{ employeeName }}</h2>
    <p>Here are your latest updates and tasks for the day.</p>
 </div></div>
 <script>
  angular.module('employeePortal', ['ngAnimate'])
    .controller('PortalController', function($scope, $timeout) {
      $scope.employeeName = "Akshatha";
      $scope.showPopup = true;
      $scope.closePopup = function() {
        $scope.showPopup = false;
    };});
 </script>
</body>
</html>
```

OUTPUT :



RESULT:

The application displays HR notifications in a popup with smooth sliding and fading animations, and each notification disappears automatically after a few seconds.

EXP NO : 28(a) **CREATE AND TEST REST API USING POSTMAN**

DATE : **(USER MANAGEMENT SYSTEM)**

AIM:

To develop a RESTful API for user management and test the API endpoints using Postman.

ALGORITHM :

Step 1: Set up a Node.js/Express server and connect it to a database (e.g., MongoDB).

Step 2: Design the user schema (fields like name, email, password).

Step 3: Create REST API endpoints for user operations:

- POST /users – Create a new user
- GET /users – Get all users
- GET /users/:id – Get single user
- PUT /users/:id – Update user
- DELETE /users/:id – Delete user

Step 4: Implement the logic for each endpoint using Express routes and Mongoose (if MongoDB used).

Step 5: Open Postman and test each endpoint by sending requests with appropriate data and headers.

Step 6: Handle responses and errors gracefully and verify correctness.

PROGRAM :

```
const express = require('express');

const app = express();

const PORT = 3000;

app.use(express.json());

let users = [

    { id: 1, name: 'Akshatha', email: 'akshatha@egmail.com' }

];

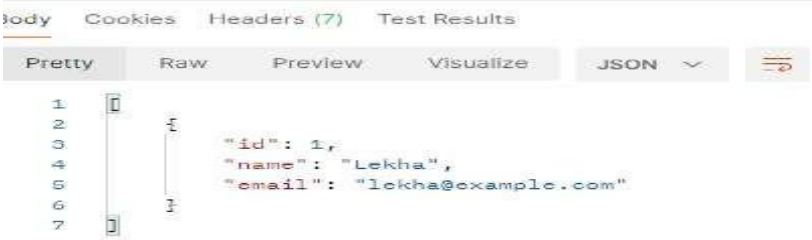
app.get('/', (req, res) => {
```

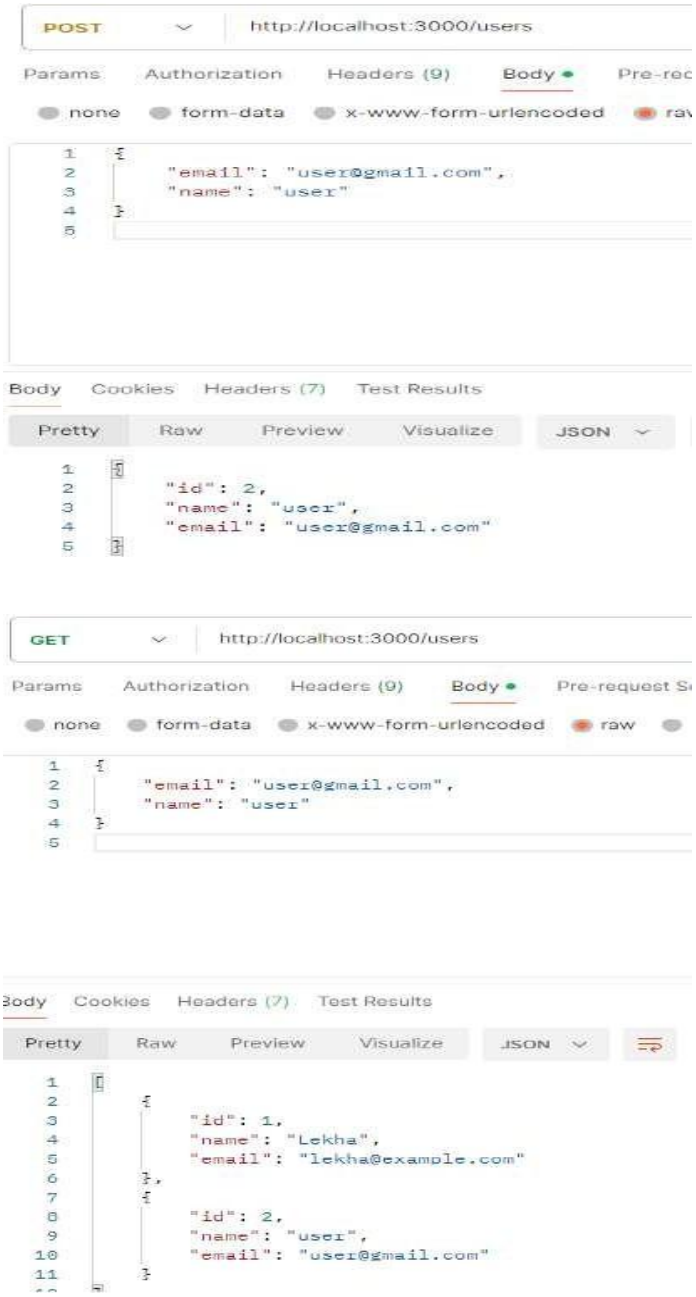
```
    res.send('User Management API is running.');
});
app.get('/users', (req, res) => {
    res.json(users);
});
app.get('/users/:id', (req, res) => {
    const user = users.find(u => u.id === parseInt(req.params.id));
    if (user) {
        res.json(user);
    } else {
        res.status(404).json({ message: 'User not found' });
    }
});
app.post('/users', (req, res) => {
    const { name, email } = req.body;
    const newUser = {
        id: users.length + 1,
        name,
        email
    };
    users.push(newUser);
    res.status(201).json(newUser);
});
app.put('/users/:id', (req, res) => {
    const { name, email } = req.body;
    const user = users.find(u => u.id === parseInt(req.params.id));
    if (user) {
        user.name = name || user.name;
        user.email = email || user.email;
```

```
        res.json(user);
    } else {
        res.status(404).json({ message: 'User not found' });
    }
});
app.delete('/users/:id', (req, res) => {
    const userIndex = users.findIndex(u => u.id === parseInt(req.params.id));
    if (userIndex !== -1) {
        users.splice(userIndex, 1);
        res.json({ message: 'User deleted successfully' });
    } else {
        res.status(404).json({ message: 'User not found' });
    }
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

OUTPUT :







RESULT :

All user-related operations (create, read, update, delete) are successfully tested and verified using Postman with the REST API working correctly.

EXP NO : 28(b) **USING ANGULAR JS SERVICES AND HTTP CALLS**

DATE : **UNIVERSITY DEPARTMENT LISTING VIA REST API**

AIM :

To modularize API calls for fetching university department data using AngularJS services and display it dynamically in a tabular format.

ALGORITHM :

Step 1: Create an AngularJS module and define a service to fetch department data.
Step 2: Use $http.get() inside the service to request department details from the REST API.
Step 3: Create a controller and inject the service into it.
Step 4: Call the service method and store the fetched data in a $scope variable.
Step 5: Display the list of departments in HTML using ng-repeat.

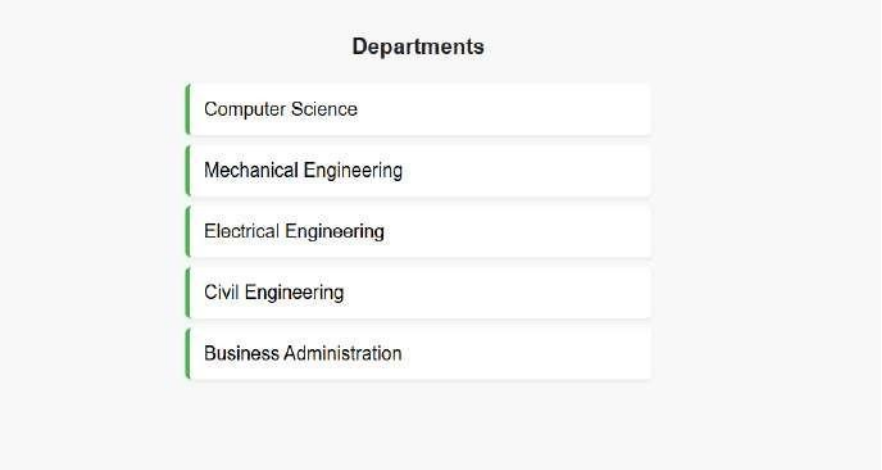
PROGRAM :

index.html:

```
<!DOCTYPE html>
<html lang=”en” ng-app=”deptApp”>
<head>
 <meta charset=”UTF-8”>
 <title>University Departments</title>
 <script src=”https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js”></script>
 <style>
  body {
   font-family: sans-serif;
   background-color: #f9f9f9;
   padding: 40px;
   text-align: center;}
  h3 {
   color: #222;
   margin-bottom: 20px;}
  ul {
   max-width: 400px;
   margin: 0 auto;
   padding: 0; }
  li {
   background-color: #ffffff;
   padding: 12px;
   margin: 8px 0;
   list-style: none;
```

```
      border-radius: 5px;
      box-shadow: 0 2px 4px rgba(0,0,0,0.05);
      border-left: 4px solid #4caf50;
      text-align: left;}
  </style>
</head>
<body ng-controller="DeptCtrl">
  <h3>Departments</h3>
  <ul><li ng-repeat="d in departments">{{ d.name }}</li></ul>
  <script>
    const app = angular.module('deptApp', []);
    app.service('DeptService', function($http) {
    this.getDepts = function() {
       return $http.get('departments.json');
};});
    app.controller('DeptCtrl', function($scope, DeptService) {
      DeptService.getDepts().then(function(response) {
        $scope.departments = response.data;
 });});});
  </script>
</body>
</html>
```

OUTPUT :



RESULT :

The application successfully loads and displays department data in a table format using AngularJS services and simulated API calls.

EXP NO : 29 **TOKEN-BASED AUTHORIZATION WITH EMAIL**

DATE : **(EMAIL VERIFICATION FLOW)**

AIM :

To implement email verification using token-based authorization. A token is sent to the user's email and verified via a secure link.

ALGORITHM :

Step 1: Register user and save with verified: false in the database.

Step 2: Generate a JWT token with user ID and expiry time.

Step 3: Send a verification email containing the token link.

Step 4: Create /verify/:token route to decode and validate the token.

Step 5: If valid, update user's status to verified: true; else return an error.

PROGRAM :

APP.JS:

```
const express = require('express');
const jwt = require('jsonwebtoken');
const nodemailer = require('nodemailer');
const cors = require('cors');
const path = require('path');
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(__dirname));
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'youremail',
    pass: 'exp 12 la env la eruka pass'
  }
```

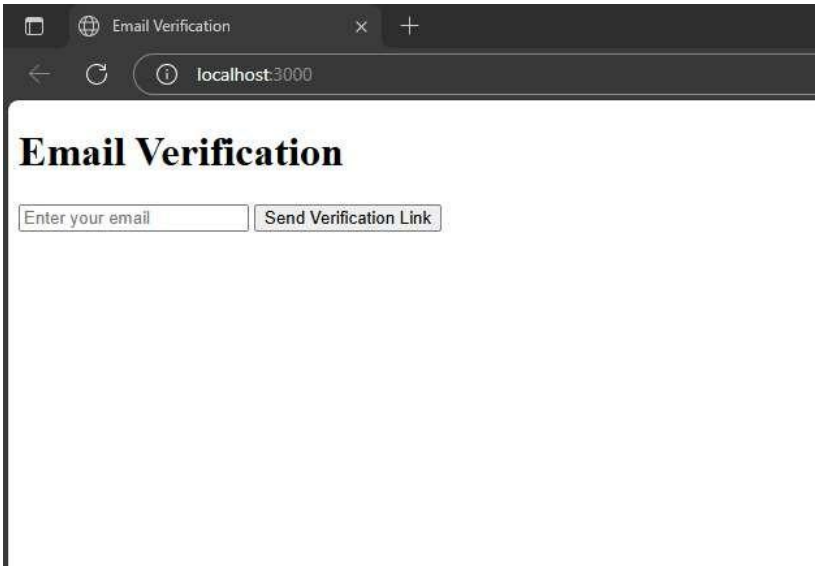
```
});
app.post('/send-token', (req, res) => {
  const { email } = req.body;
  const token = jwt.sign({ email }, 'secretKey', { expiresIn: '10m'
  }); const link = `http://localhost:3000/verify/${token}`;
  transporter.sendMail({
    from:
    'yourgmail@gmail.com', to:
    email,
    subject: 'Verify Your Email',
    html: `<h2>Email Verification</h2><p><a href="${link}">Click here to verify</a></p>`
  }, (err, info) => {
    if (err) {
      console.error(err);
      return res.status(500).send('Failed to send email');
    }
    res.send('Verification email sent to ' + email);
  });
});
app.get('/verify/:token', (req, res) => {
  try {
    const data = jwt.verify(req.params.token, 'secretKey');
    res.send(`<h2>Email <i>${data.email}</i> verified successfully!</h2>`);
  } catch {
    res.status(400).send('<h2>Invalid or expired verification link.</h2>');
  }
});
app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

INDEX.HTML:

```
<!DOCTYPE html>
<html lang=”en”>
<head>
  <meta charset=”UTF-8” />
  <title>Email Verification</title>
</head>
<body>
  <h1>Email Verification</h1>
  <form id=”verifyForm”>
    <input type=”email” id=”email” placeholder=”Enter your email” required />
    <button type=”submit”>Send Verification Link</button>
  </form>
  <p id=”responseMsg”></p>
  <script>
    document.getElementById(‘verifyForm’).addEventListener(‘submit’, async € => {
      e.preventDefault();
      const email =
      document.getElementById(‘email’).value; const res =
      await fetch(‘/send-token’, {
        method: ‘POST’,
        headers: { ‘Content-Type’: ‘application/json’ },
        body: JSON.stringify({ email })
      });
      const msg = await res.text();
      document.getElementById(‘responseMsg’).innerText = msg;
    });
  </script>
</body>
</html>
```

OUTPUT :

1. Verification Page



2. Mail Page



RESULT :

The client-side validation using HTML5 attributes ensures correct email format, minimum password length, and valid 10-digit phone number. This enhances user input accuracy before form submission.

EXP NO :30(a) **SECURE LOGIN FLOW USING EMAIL VERIFICATION AND JWT**

DATE :

AIM :

To build a secure login system where a user receives an email verification link, and upon successful verification, receives a JWT token to access protected routes.

ALGORITHM :

Step 1: Start the process.
Step 2: Register user and generate a JWT verification token.
Step 3: Send verification link to user's email.
Step 4: On clicking the link, verify the token and mark user as verified.
Step 5: Issue a new JWT token for login and allow access to protected routes.

PROGRAM:

APP.JS

```
const express = require("express");
const dotenv = require("dotenv");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const nodemailer = require("nodemailer");
const cors = require('cors');
dotenv.config();
const app = express();
app.use(express.json());
app.use(cors());
const users = {};
const transporter = nodemailer.createTransport({
  service: "Gmail",
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS
  }
});
app.post("/register", async (req, res) => {
  const { email, password } = req.body;
  try {
    if (users[email]) return res.status(400).json({ msg: "User already exists" });
    const hashed = await bcrypt.hash(password, 10);
    users[email] = { password: hashed, isVerified: false };
    const token = jwt.sign({ email }, process.env.JWT_SECRET, { expiresIn: "1h" });
```

```
    const link = `${process.env.CLIENT_URL}/verify/${token}`;
    try {
      await transporter.sendMail({
        to: email,
        subject: “Verify your Email”,
        html: `<p>Click to verify your email: <a href=”${link}”>${link}</a></p>`
      });
    } catch (   mailer) {
      console.error(“ Email send failed:”,    mailer); // <- Print email error
      return res.status(500).json({ msg: “Email sending failed” });
    }
    res.json({ msg: “Verification email sent” });
  } catch (err) {
    console.error(“ Registration error:”, err); // <- Print backend error
    res.status(500).json({ msg: “Internal server error” });
  }
});
app.get(“/verify/:token”, (req, res) => {
  try {
    const { email } = jwt.verify(req.params.token, process.env.JWT_SECRET);
    if (!users[email]) return res.status(400).send(“User not found”);
    users[email].isVerified = true;
    const accessToken = jwt.sign({ email }, process.env.JWT_SECRET, {
      expiresIn: process.env.JWT_EXPIRES_IN
    });
    res.send(`
      <h2>Email verified </h2>
      <p>Your access token:</p>
      <code>${accessToken}</code>
    `);
  } catch (err) {
    res.status(400).send(“Invalid or expired token”);
  }
});
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(“ “)[1];
  if (!token) return res.status(401).json({ msg: “No token” });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.email = decoded.email;
    next();
  } catch {
   res.status(401).json({ msg: “Invalid token” });}}
```

```
app.get("/dashboard", authMiddleware, (req, res) => {
  const user = users[req.email];
  if (!user || !user.isVerified)
    return res.status(403).json({ msg: "Email not verified" });
  const tokenExpiry = new Date(Date.now() + 60 * 60 * 1000).toISOString(); // +1 hour
  res.json({
    msg: `Welcome ${req.email}, to your dashboard! `,
    user: {
      email: req.email,
      isVerified: user.isVerified,
    },
    tokenInfo: {
      validUntil: tokenExpiry,
      type: "Bearer"
    },
    lastLogin: new Date().toLocaleString()
  });
});
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

INDEX.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Secure Login Test</title>
    <style>
        body {
            font-family: Arial;
            max-width: 500px;
            margin: auto;
            padding: 20px; }
        input,button {
            display: block;
            margin: 10px 0;
            padding: 10px;
            width: 100%;}
            code {
                    background: #eee;
                    padding: 5px;
                    display: block; }
    </style>
```

```
</head>
<body>
  <h1> Register & Get Verified</h1>
  <form id="registerForm">
    <input type="email" id="email" placeholder="Enter email" required />
    <input type="password" id="password" placeholder="Enter password" required />
    <button type="submit">Register</button>
  </form>
  <h2> Use JWT Token</h2>
  <textarea id="tokenInput" rows="4" placeholder="Paste JWT token from verification page"></textarea>
  <button onclick="accessDashboard()">Access Dashboard</button>
  <div id="result"></div>
  <script>
    const API_URL = "http://localhost:5000";
    document.getElementById("registerForm").addEventListener("submit", async € => {
      e.preventDefault();
      const email = document.getElementById("email").value;
      const password = document.getElementById("password").value;
      try {
        const res = await fetch(`${API_URL}/register`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ email, password })
        });
        const data = await res.json();
        document.getElementById("result").innerHTML =
`<p>${data.msg}</p><p>Check your email for verification link.</p>`;
      } catch (err) {
        document.getElementById("result").innerText = "Registration failed.";
      }
    });
    async function accessDashboard() {
      const token = document.getElementById("tokenInput").value.trim();
      if (!token) return alert("Please paste a valid JWT token.");
      try {
        const res = await fetch(`${API_URL}/dashboard`, {
          headers: {
            Authorization: `Bearer ${token}`
          }
        });
        const data = await res.json();
        document.getElementById("result").innerHTML = `
```

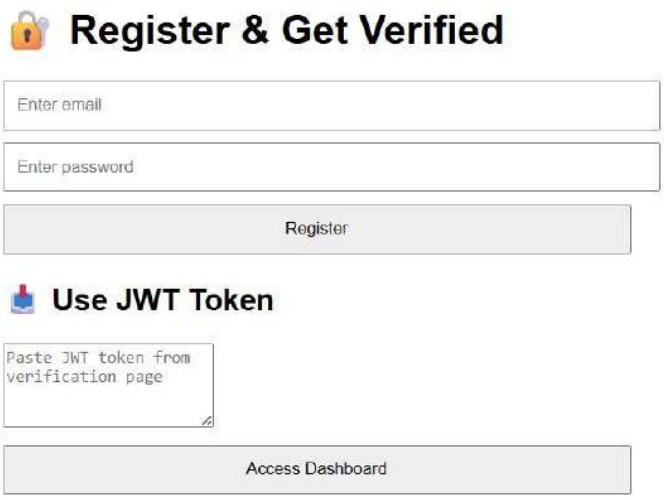
```
  <h3>Dashboard Access Granted</h3>
  <pre>${JSON.stringify(data, null, 2)}</pre>
          } catch (err) {
             document.getElementById("result").innerText = "Could not access dashboard.";
          }
       }
    </script>
</body>
</html>
```

.env

```
PORT=5000
JWT_SECRET=supersecretkey
JWT_EXPIRES_IN=1h
EMAIL_USER=lekhapandic@gmail.com
EMAIL_PASS=xxltblgbsoyfucqd
CLIENT_URL=http://localhost:5000
```

OUTPUT:

1. **Home Page**



2. Registartion Page



RESULT:

The secure login system with email verification and JWT was successfully implemented. Verified users can now access protected routes using the issued token.

EXP NO : 30(b) **IMPLEMENTING I18N WITH ANGULARJS – MULTI-**

DATE : **LANGUAGE STUDENT PORTAL INTERFACE**

AIM :

To enable multi-language support in a student portal using AngularJS and angular-translate.

ALGORITHM :

Step 1: Create an AngularJS module with pascalprecht.translate as a dependency.
Step 2: Configure $translateProvider with translations for all supported languages.
Step 3: Set a default language using $translateProvider.preferredLanguage().
Step 4: Create a controller with a function to change the current language using $translate.use(lang).
Step 5: Use {{ 'KEY' | translate }} in HTML to dynamically display the translated text.

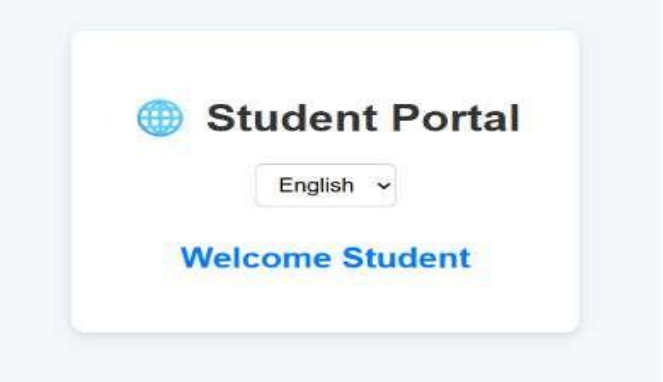
PROGRAM:

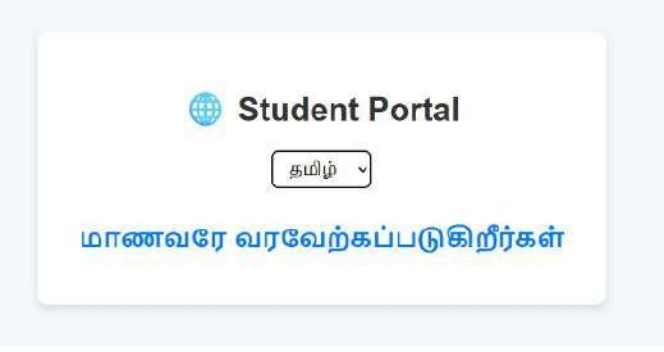
index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="langApp">
<head>
  <meta charset="UTF-8">
  <title>Multi-language Student Portal</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
 <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f7f9;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
      margin: 0;}
  </style>
</head>
<body ng-controller="LangCtrl">
<div class="container">
    <h1>Student Portal</h1>
    <select ng-model="lang" ng-change="changeLang(lang)">
      <option value="en">English</option>
      <option value="ta">தமிழ்</option>
    </select>
    <h2>{{ 'GREETING' | translate }}</h2></div>
<script>
    angular.module('langApp', ['pascalprecht.translate'])
```

```
    .config(function($translateProvider) {
      $translateProvider.translations('en', {
        GREETING: "Welcome Student"});
      $translateProvider.translations('ta', {
        GREETING: "மாணவரே வேரவற்கப்படுகிறீேக்கள்"});
      $translateProvider.preferredLanguage('en');})
    .controller('LangCtrl', function($scope, $translate) {
      $scope.lang = 'en';
      $scope.changeLang = function(lang) {
        $translate.use(lang);
    };});
  </script>
</body>
</html>
```

OUTPUT :





RESULT:

The student portal interface successfully switches between English, Hindi, and Tamil, updating the welcome message and labels in real time without reloading the page.

EX NO: 31 DATE:	DB CONNECTIVITY IN MERN STACK

AIM:

To connect a MongoDB database with Node.js (Express) in a Mern Stack app.

ALGORITHM:

1. Start the server using Node.js and Express.
2. Connect to MongoDB database using Mongoose with a connection string.
3. Define schema and model to structure how data will be stored in MongoDB.
4. Create API routes in Express to insert, read, update, or delete data.
5. Send requests from client (React/Postman) to API routes and get responses from the database.

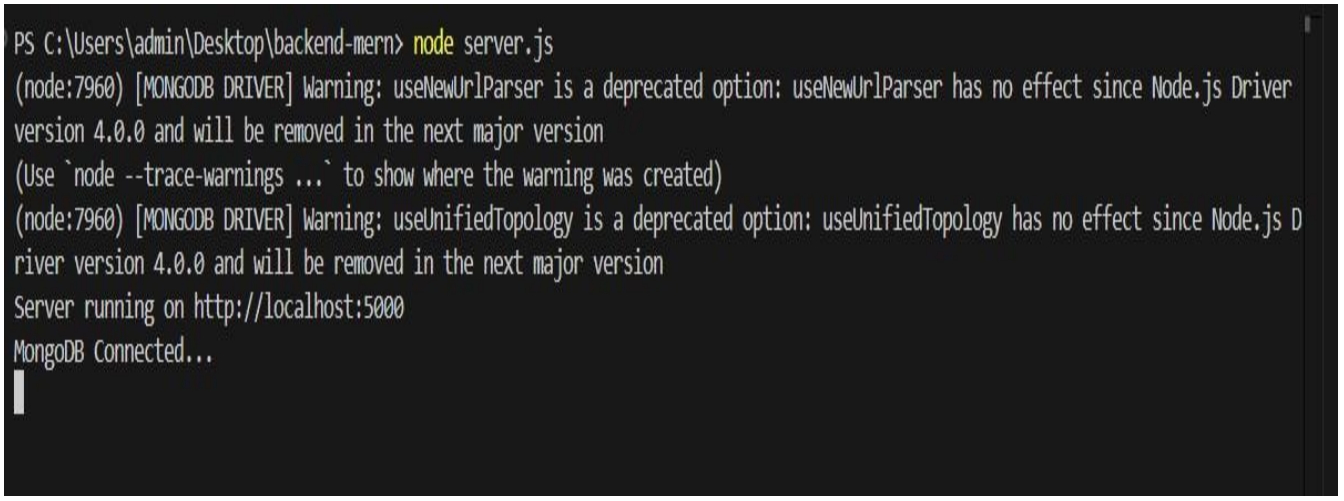
PROGRAM:

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
mongoose.connect("mongodb://127.0.0.1:27017/mydb", {
useNewUrlParser: true,
useUnifiedTopology: true, })
.then(() => console.log("MongoDB Connected..."))
catch(err => console.error("DB Connection Error:", err));
const userSchema = new mongoose.Schema({
 name: String,
 email: String,
 age: Number });
const User = mongoose.model("User", userSchema);
app.get("/", (req, res) => {
  res.send("API is running...");
});
```

```
app.post("/api/users", async (req, res) => {
try {
        const user = new User(req.body);
        const savedUser = await user.save();
        res.json(savedUser);
         } catch (err) {
           res.status(500).json({ error: err.message }); } });
app.get("/api/users", async (req, res) => {
 try {
           const users = await User.find();
           res.json(users);
         } catch (err) {
           res.status(500).json({ error: err.message }); } });
       const PORT = 5000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

OUTPUT:

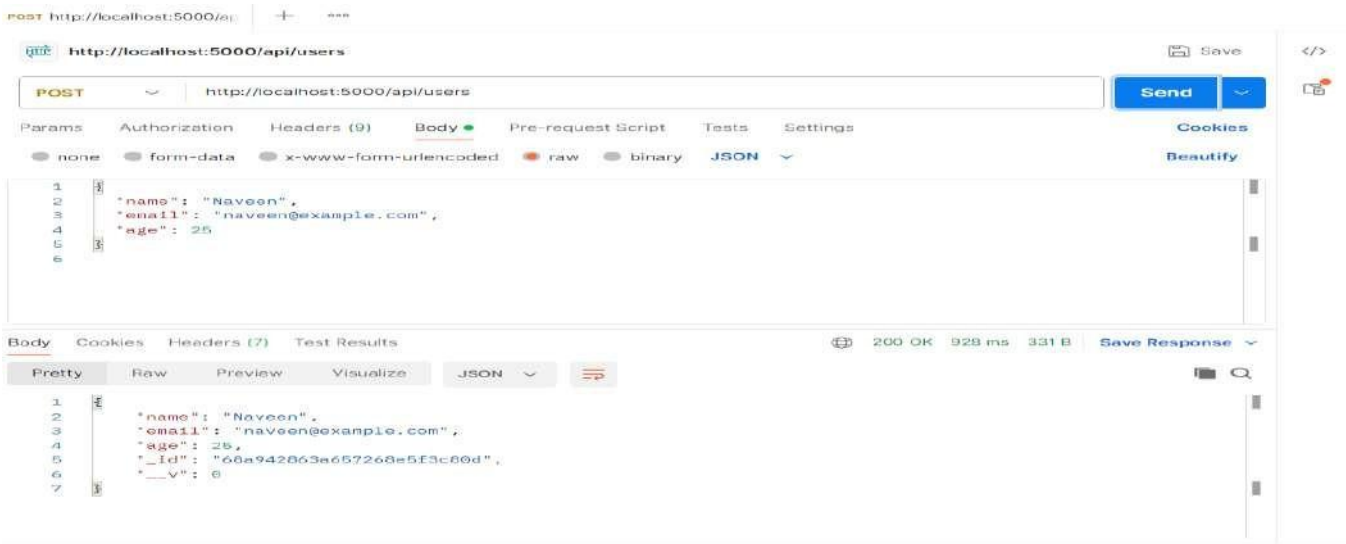
1. Start the server



2. Verify server is reachable



3. Verify DB



RESULT:

Hence the database connectivity is done successfully.

EX NO: 32 DATE:	CRUD OPERATION USING MONGODB

AIM:

To perform Create, Read, Update, Delete operation using MongoDB in a Node.js app

ALGORITHM:

1. Start MongoDB service (mongod).

2. Initialize Node.js project and install dependencies (express, mongoose, body-parser, cors).

3. Create Express server (server.js).

4. Connect Express server to MongoDB using Mongoose.

5. Define a Schema and Model (e.g., Student).

6. Implement Create route (POST /students) → insert record.

7. Implement Read All route (GET /students) → fetch all records.

8. Implement Update route (PUT /students/:id) → modify existing record.

9. Implement Delete route (DELETE /students/:id) → remove record.

PROGRAM:

```
const express = require("express");
const mongoose = require("mongoose");
const bodyParser = require("body-parser");
const app = express();
app.use(bodyParser.json());
mongoose.connect("mongodb://127.0.0.1:27017/merncrud", {
useNewUrlParser: true,
useUnifiedTopology: true, });
```

```
const db = mongoose.connection;

db.on(“error”, console.error.bind(console, “MongoDB connection error:”));

db.once(“open”, () => console.log(“MongoDB Connected”));
```

```
const StudentSchema = new mongoose.Schema({
name: String,
age: Number,
course: String, });
const Student = mongoose.model("Student", StudentSchema);
app.post("/students", async (req, res) => {
try {
const student = new Student(req.body);
await student.save();
res.status(201).json(student);
} catch (err) {
res.status(400).json({ error: err.message }); } });
app.get("/students", async (req, res) => {
const students = await Student.find();
res.json(students); });
app.get("/students/:id", async (req, res) => {
try {
const student = await Student.findById(req.params.id);
if (!student) return res.status(404).json({ error: "Not Found" });
res.json(student);
} catch (err) {
res.status(400).json({ error: err.message }); } });
app.put("/students/:id", async (req, res) => {
try {
const student = await Student.findByIdAndUpdate(req.params.id, req.body, { new: true, });
```

```
if (!student) return res.status(404).json({ error: “Not Found” });

res.json(student);

} catch (err) {

res.status(400).json({ error: err.message }); } });

app.delete(“/students/:id”, async (req, res) => {

try {

const student = await Student.findByIdAndDelete(req.params.id);

if (!student) return res.status(404).json({ error: “Not Found” });

res.json({ message: “Deleted Successfully” });

} catch (err) {

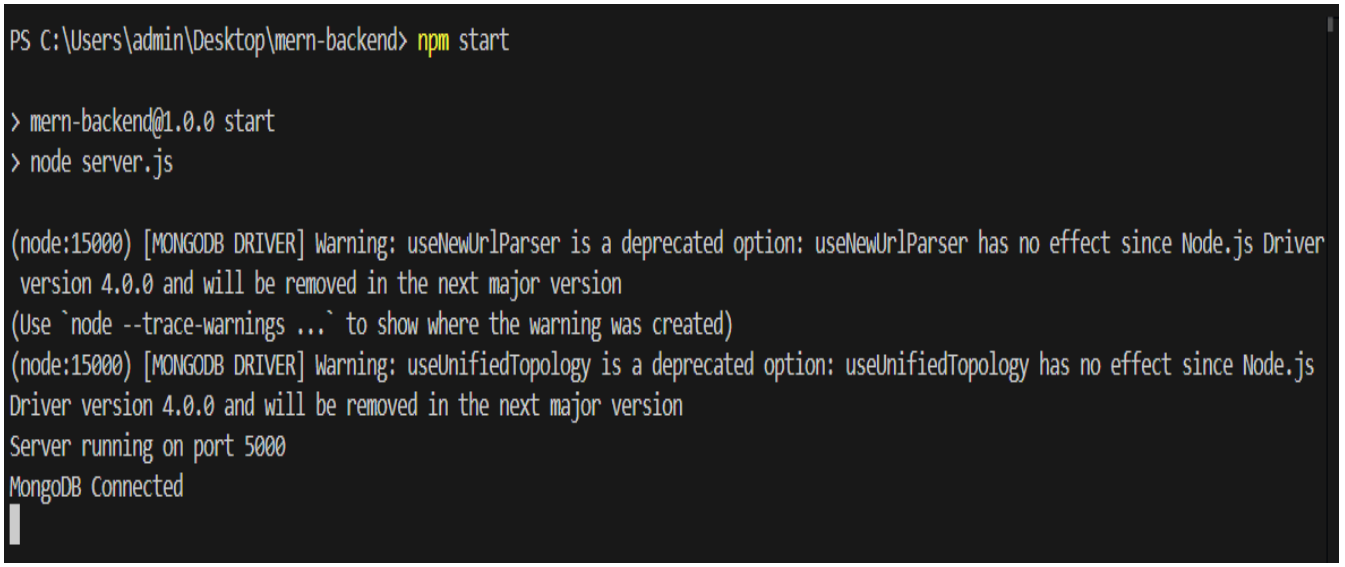
res.status(400).json({ error: err.message }); }  });

const PORT = 5000;

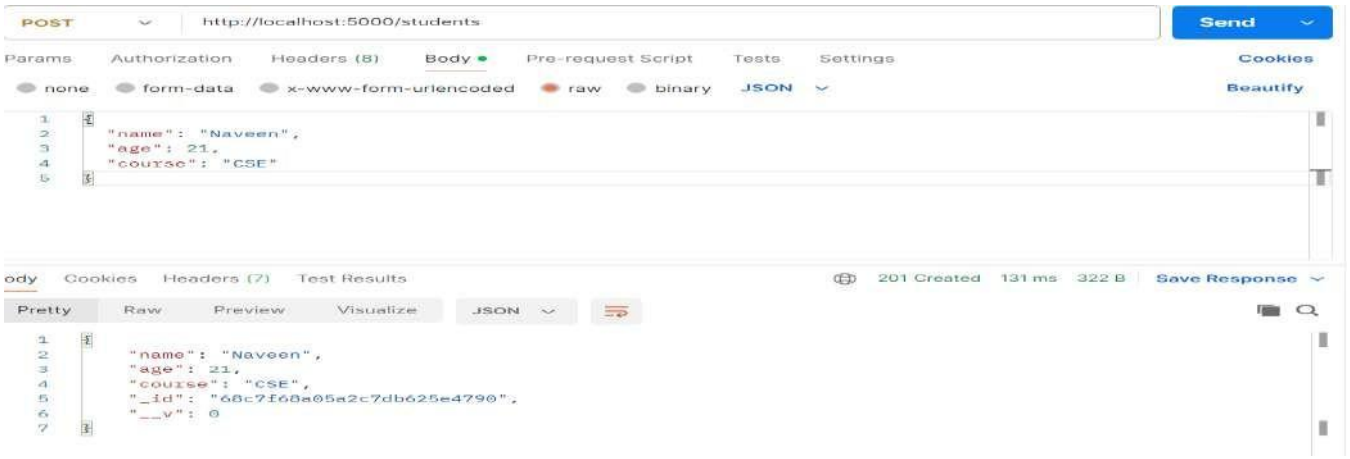
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

OUTPUT:

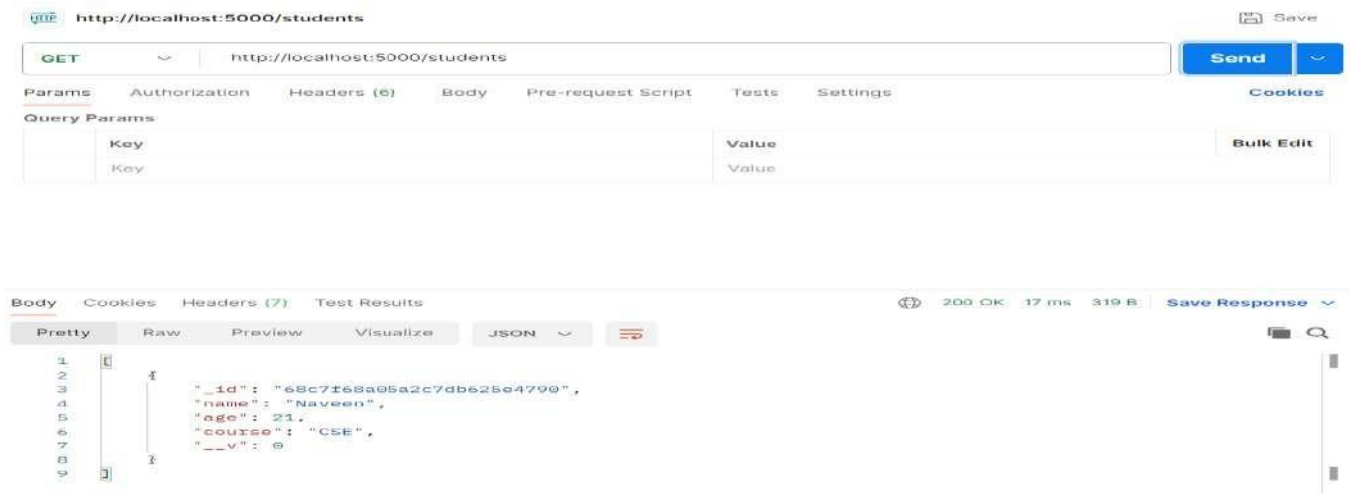
1. MONGODB Connectivity



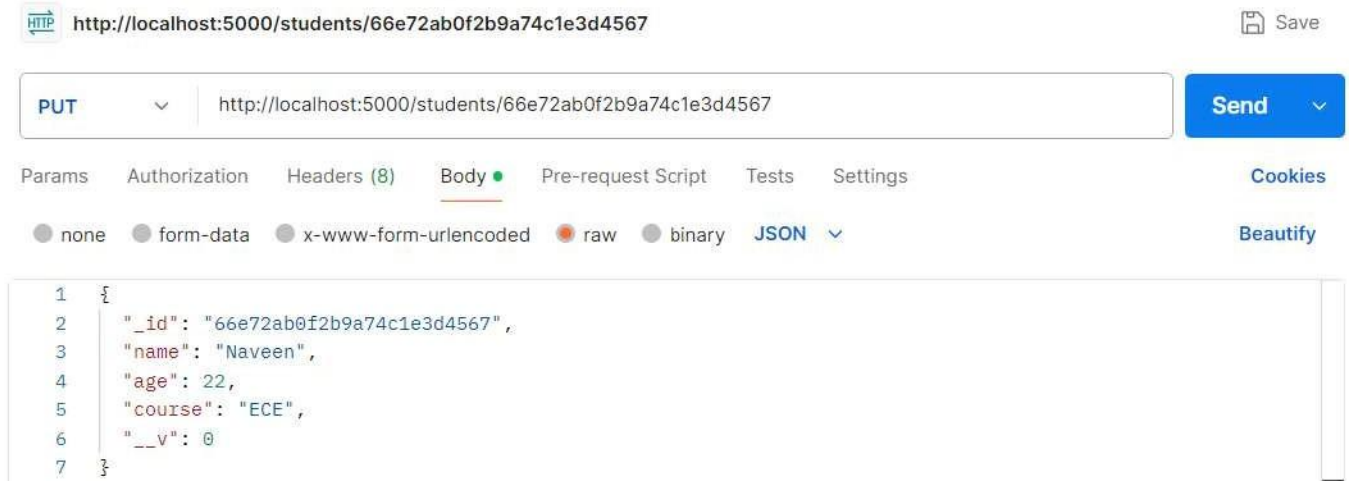
2. Create Student (POST)



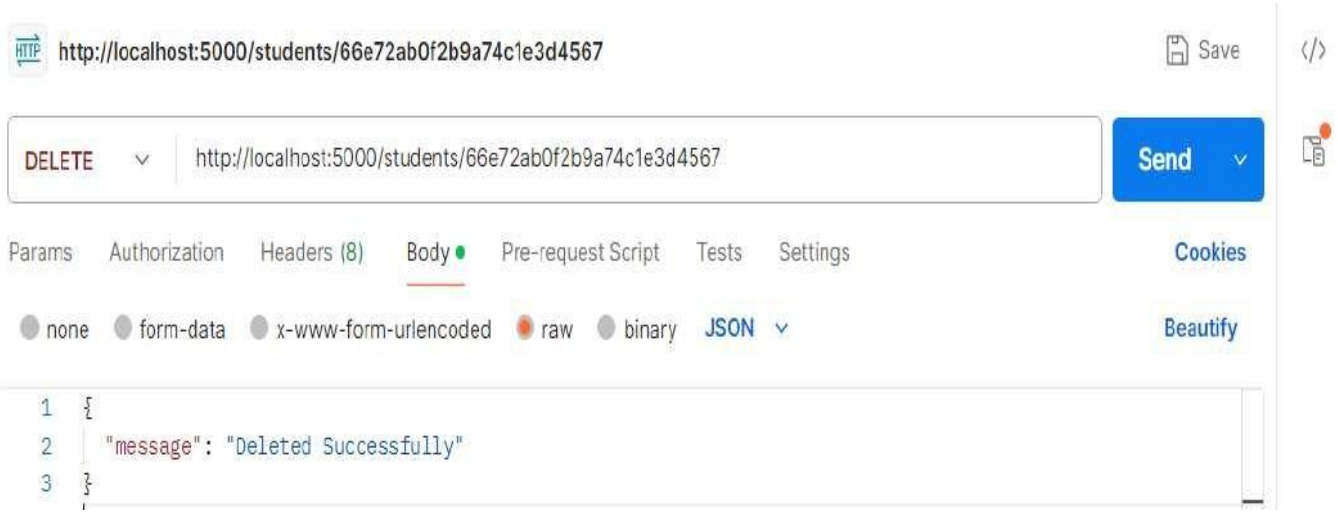
3. Read All Students (GET)



4. Update Student (PUT)



5. Delete Student (DELETE)



RESULT:

Hence the CRUD operation is performed successfully.